

Invert Trends

John Grady

2025-10-21

R Markdown

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE

# use R Project GitHub/Great-Smoky-Mountains-Vital-Trends/Great-Smoky-Mountains-Vital-Trends.Rproj
# -----
# -----
# Define root paths (user-specific)
# -----
github_path <- "~/Documents/GitHub/Great-Smoky-Mountains-Vital-Trends"
drive_path  <- "/Users/jgradym/Library/CloudStorage/GoogleDrive-jgradym@gmail.com/Shared drives/GRSM_CES/"

# Datasets
ncbi <- readxl::read_xlsx(file.path(drive_path, "Maine/Data/Aquatics_Macroinverts/NCBI_EPT/NCBI_taxa.xlsx"))
inverts <- readr::read_csv(file.path(drive_path, "Maine/Data/Aquatics_Macroinverts/SummaryData/SpecimenData.csv"))

## Rows: 23923 Columns: 17
## -- Column specification -----
## Delimiter: ","
## chr  (12): LOC_NAME, Sample_Code, Lab_Scientific_Name, Lab_Phylum, Lab_Order...
## dbl  (3): Count, LAT, LON
## dtm   (1): EventDate
## date  (1): Start_Date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
inverts_original = readr::read_csv(file.path(drive_path, "Maine/Data/Aquatics_Macroinverts/SummaryData/SpecimenData.csv"))

## Rows: 24182 Columns: 10
## -- Column specification -----
## Delimiter: ","
## chr  (7): LOC_NAME, Sample_Code, Lab_Scientific_Name, Lab_Phylum, Lab_Order,...
## dbl  (1): Count
```

```

## dtm (1): EventDate
## date (1): Start_Date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
# Spatial layers (consistent format)
grsm_watershed = st_read(file.path(drive_path, "Maine/Spatial/Watershed/GRSM_watershed.gpkg"))[2]

## Reading layer `GRSM_watershed' from data source
##   `/Users/jgradym/Library/CloudStorage/GoogleDrive-jgradym@gmail.com/Shared drives/GRSM_CESU/Maine/S
##   using driver `GPKG'
## Simple feature collection with 9 features and 4 fields
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -84.01389 ymin: 35.42967 xmax: -82.99804 ymax: 35.84245
## Geodetic CRS:   NAD83

grsm_streams = st_read(file.path(drive_path, "Maine/Spatial/Streams/GRSM_streams.gpkg"))

## Reading layer `GRSM_streams' from data source
##   `/Users/jgradym/Library/CloudStorage/GoogleDrive-jgradym@gmail.com/Shared drives/GRSM_CESU/Maine/S
##   using driver `GPKG'
## Simple feature collection with 9281 features and 1 field
## Geometry type: MULTILINESTRING
## Dimension:      XY
## Bounding box:   xmin: 226483.3 ymin: 3922474 xmax: 319548.1 ymax: 3970539
## Projected CRS:  NAD83 / UTM zone 17N

grsm_border <- sf::st_read(file.path(drive_path, "Maine/Spatial/BOUNDARY_LN/BOUNDARY_LN.shp"))[6] %>%
  sf::st_transform(sf::st_crs(grsm_watershed))

## Reading layer `BOUNDARY_LN' from data source
##   `/Users/jgradym/Library/CloudStorage/GoogleDrive-jgradym@gmail.com/Shared drives/GRSM_CESU/Maine/S
##   using driver `ESRI Shapefile'
## Simple feature collection with 19 features and 12 fields
## Geometry type: MULTILINESTRING
## Dimension:      XY
## Bounding box:   xmin: 226810.1 ymin: 3923747 xmax: 319130.8 ymax: 3968555
## Projected CRS:  NAD83 / UTM zone 17N

# =====
# Augment files
# =====

# add genus to NCBI from species name
ncbi$Genus <- stringr::word(ncbi$Species, 1, sep = " ")

# Get average ncbi value of species per genus
ncbi_genus <- ncbi %>%
  group_by(Genus) %>%
  summarise(
    Order      = first(str_split_fixed(na.omit(Order), ":", 2)[, 1]), # keep order - first entry per
    genus_ncbi = mean(NCBI, na.rm = TRUE),
    n_species  = n(),
    n_scored   = sum(!is.na(NCBI)),
    .groups = "drop"

```

```

)

# Check original invert file for duplicates

dups <- inverts_original %>%
  mutate(Year = as.integer(format(EventDate, "%Y"))) %>%
  group_by(across(everything())) %>% # group by every column, only duplicates will appear twice
  filter(n() > 1) %>% #within each group, n() counts how many rows there are. Filter to more than 1
  ungroup() %>%
  arrange(across(everything())) #sets output so that duplicates are next to each other - note if not, i
dups #492 rows of duplicates - ie, 246 rows have a duplicate

## # A tibble: 492 x 11
##   LOC_NAME EventDate      Start_Date Sample_Code Lab_Scientific_Name Count
##   <chr>      <dtm>          <date>      <chr>      <chr>          <dbl>
## 1 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Ablabesmyia mallocc~    9
## 2 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Ablabesmyia mallocc~    9
## 3 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Ablabesmyia mallocc~   14
## 4 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Ablabesmyia mallocc~   14
## 5 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Acentrella ampla      7
## 6 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Acentrella ampla      7
## 7 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Acentrella ampla      9
## 8 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Acentrella ampla      9
## 9 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Acentrella ampla     13
## 10 Abrams ~ 2017-06-12 11:36:31 2019-07-29 ABAB012019 Acentrella ampla     13
## # i 482 more rows
## # i 5 more variables: Lab_Phylum <chr>, Lab_Order <chr>, Lab_Family <chr>,
## #   Lab_Species <chr>, Year <int>

dups %>%
  count(Year, name = "n_duplicates") %>%
  arrange(Year)

## # A tibble: 2 x 2
##   Year n_duplicates
##   <int>      <int>
## 1 2016          4
## 2 2017        488

# Inverts with locations

# -----
# Whitespace cleanup
# - Trim only character columns; leave numeric/integer columns unchanged
# -----

inverts <- inverts %>%
  mutate(across(where(is.character), str_squish))

# -----
# Add helpful columns - Stream location, site, Genus, and year
# -----

inverts$Location <- stringr::word(inverts$LOC_NAME, 1, sep = ",")
inverts$Site <- stringr::word(inverts$LOC_NAME, 2, sep = ",")
inverts$Genus <- stringr::word(inverts$Lab_Scientific_Name, 1, sep = " ")
inverts$Year <- as.numeric(stringr::word(inverts$Start_Date, 1, sep = "-"))

```

```

# Add ncbi
inverts <- invert %>%
  left_join(ncbi_genus %>% dplyr::select(Genus, genus_ncbi), by = "Genus")

# -----
# ----- Table checks -----
# -----
#When sampled at at given location

time_table_samplecodes <- invert %>%
  filter(!is.na(Start_Date), !is.na(Sample_Code)) %>%
  transmute(
    Year = year(Start_Date),
    Month = factor(month(Start_Date), levels = 1:12, labels = month.abb),
    Sample_Code
  ) %>%
  distinct(Year, Month, Sample_Code) %>% # one count per unique sample code in a Year-Month
  count(Year, Month, name = "n_samplecodes") %>% # totals per Year-Month
  pivot_wider(
    names_from = Month,
    values_from = n_samplecodes,
    values_fill = 0
  ) %>%
  arrange(Year) %>%
  relocate(Year, all_of(month.abb))

time_table_samplecodes %>% print(n = 27)

```

```

## # A tibble: 26 x 13
##   Year Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
##   <dbl> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1 1984 0 0 0 0 0 0 0 1 0 0 0 0
## 2 1986 0 0 0 0 0 0 1 0 0 0 0 0
## 3 1989 0 0 0 0 0 0 2 0 0 0 0 0
## 4 1990 5 0 0 2 0 0 2 0 0 2 0 0
## 5 1991 0 0 0 0 0 0 1 0 0 0 0 0
## 6 1992 0 0 0 0 0 0 0 1 0 0 5 0
## 7 1993 0 0 0 0 0 7 6 0 0 0 0 0
## 8 1994 0 0 0 0 0 20 9 6 11 0 0 0
## 9 1995 0 0 0 0 1 8 10 15 7 0 0 0
## 10 1996 0 0 0 0 0 13 11 16 17 1 1 1
## 11 1997 1 1 1 1 1 9 14 12 9 0 0 0
## 12 1998 0 0 0 0 0 9 9 10 7 0 0 0
## 13 1999 0 0 0 0 0 8 8 8 6 0 0 0
## 14 2000 0 0 0 0 0 6 7 9 0 0 0 0
## 15 2001 0 0 0 0 0 7 11 8 6 1 0 0
## 16 2002 0 0 0 0 0 6 9 6 0 0 0 0
## 17 2003 0 0 0 0 0 9 10 3 2 1 0 0
## 18 2014 0 0 0 0 0 2 5 1 0 0 0 0
## 19 2015 0 0 0 0 0 2 5 0 0 0 0 0
## 20 2016 0 0 0 0 0 4 3 1 0 0 0 0
## 21 2017 0 0 0 0 0 3 3 1 0 0 0 0
## 22 2019 0 0 0 0 0 3 3 2 0 0 0 0

```

```
## 23 2020    0    0    0    0    0    2    3    2    0    0    0    0
## 24 2021    0    0    0    0    0    3    5    0    0    0    0    0
## 25 2022    0    0    0    0    0    3    4    1    0    0    0    0
## 26 2023    0    0    0    0    0    2    4    1    0    0    0    0
```

```
# same but by location name
time_table_location <- invert %>%
  filter(!is.na(Start_Date), !is.na(LOC_NAME)) %>%
  transmute(
    Year = year(Start_Date),
    Month = factor(month(Start_Date), levels = 1:12, labels = month.abb),
    LOC_NAME
  ) %>%
distinct(Year, Month, LOC_NAME) %>% # one count per unique sample code in a Year-Month
count(Year, Month, name = "n_loc_names") %>% # totals per Year-Month
pivot_wider(
  names_from = Month,
  values_from = n_loc_names,
  values_fill = 0
) %>%
arrange(Year) %>%
relocate(Year, all_of(month.abb))

time_table_location %>% print(n = 27)
```

```
## # A tibble: 27 x 13
##   Year Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
##   <dbl> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1 1984    0    0    0    0    0    0    0    1    0    0    0    0
## 2 1986    0    0    0    0    0    0    1    0    0    0    0    0
## 3 1989    0    0    0    0    0    0    1    0    0    0    0    0
## 4 1990    4    0    0    2    0    0    2    0    0    2    0    0
## 5 1991    0    0    0    0    0    0    1    0    0    0    0    0
## 6 1992    0    0    0    0    0    0    0    1    0    0    5    0
## 7 1993    0    0    0    0    0    7    6    0    0    0    0    0
## 8 1994    0    0    0    0    0    19   9    6   11    0    0    0
## 9 1995    0    0    0    0    1    8   10   15    7    0    0    0
## 10 1996    0    0    0    0    0    12   11   16   16    1    1    1
## 11 1997    1    1    1    1    1    9   14   12    9    0    0    0
## 12 1998    0    0    0    0    0    9    9   10    7    0    0    0
## 13 1999    0    0    0    0    0    8    8    8    6    0    0    0
## 14 2000    0    0    0    0    0    6    7    9    0    0    0    0
## 15 2001    0    0    0    0    0    7   11    8    6    1    0    0
## 16 2002    0    0    0    0    0    6    9    6    0    0    0    0
## 17 2003    0    0    0    0    0    9   10    3    2    1    0    0
## 18 2005    0    0    0    0    0    0    0    1    0    0    0    0
## 19 2014    0    0    0    0    0    2    5    1    0    0    0    0
## 20 2015    0    0    0    0    0    2    5    0    0    0    0    0
## 21 2016    0    0    0    0    0    4    3    1    0    0    0    0
## 22 2017    0    0    0    0    0    3    3    1    0    0    0    0
## 23 2019    0    0    0    0    0    3    3    2    0    0    0    0
## 24 2020    0    0    0    0    0    2    3    2    0    0    0    0
## 25 2021    0    0    0    0    0    3    5    0    0    0    0    0
## 26 2022    0    0    0    0    0    3    4    1    0    0    0    0
## 27 2023    0    0    0    0    0    2    4    1    0    0    0    0
```

```

# by site, month and year
time_table2 <- invert %>%
  dplyr::filter(!is.na(Start_Date)) %>%
  dplyr::distinct(
    LOC_NAME,
    Year = lubridate::year(Start_Date),
    Month = lubridate::month(Start_Date, label = TRUE, abbr = TRUE),
    Sample_Code
  ) %>% # each Sample_Code counts once within LOC_NAME x Year x Month
  dplyr::count(LOC_NAME, Year, Month, name = "n_samples") %>%
  tidyr::pivot_wider(
    names_from = Month, # creates Jan..Dec columns
    values_from = n_samples,
    values_fill = 0
  ) %>%
  dplyr::arrange(LOC_NAME, Year) %>%
  dplyr::relocate(LOC_NAME, Year, dplyr::all_of(month.abb))
time_table2 %>% print(n = 36)

```

```

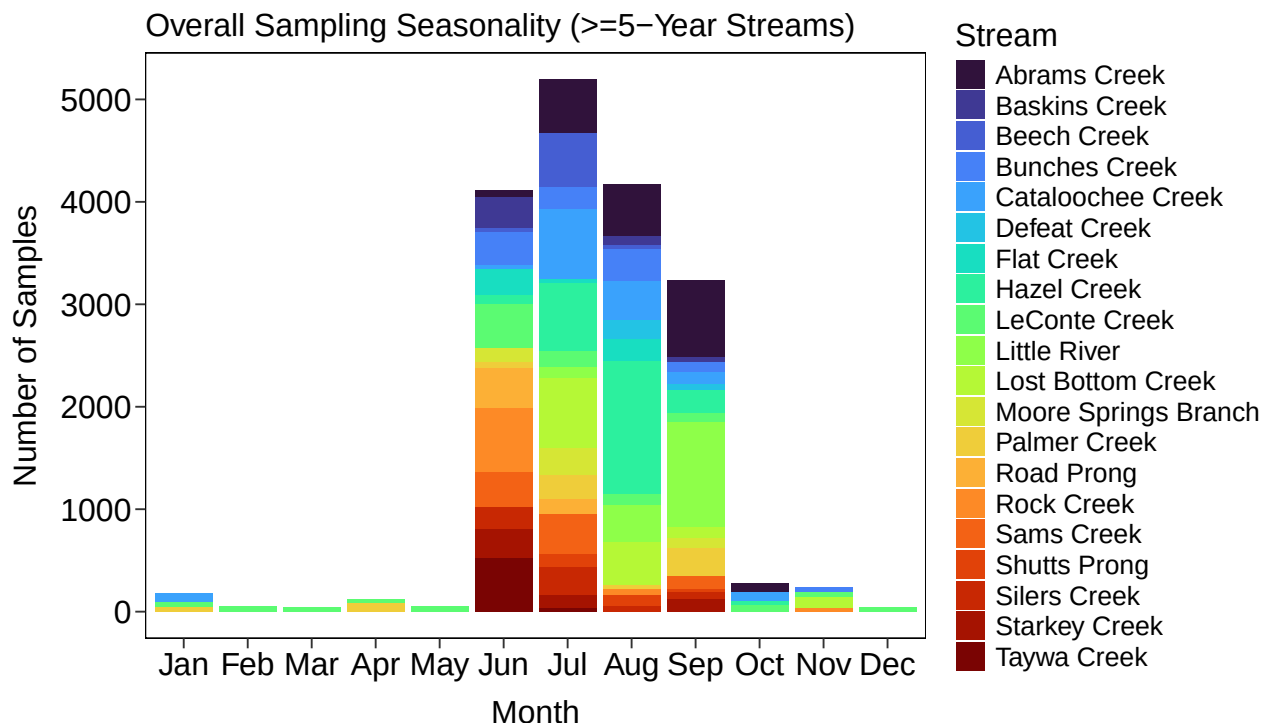
## # A tibble: 435 x 14
##   LOC_NAME      Year  Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct
##   <chr>      <dbl> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1 Abrams Cre~ 1994     0     0     0     0     0     0     0     0     1     0
## 2 Abrams Cre~ 1995     0     0     0     0     0     0     0     0     1     0
## 3 Abrams Cre~ 1996     0     0     0     0     0     0     0     0     1     0
## 4 Abrams Cre~ 1997     0     0     0     0     0     0     0     0     1     0
## 5 Abrams Cre~ 1998     0     0     0     0     0     0     0     0     1     0
## 6 Abrams Cre~ 1999     0     0     0     0     0     0     0     1     0     0
## 7 Abrams Cre~ 2000     0     0     0     0     0     0     0     1     0     0
## 8 Abrams Cre~ 2001     0     0     0     0     0     0     0     0     0     1
## 9 Abrams Cre~ 2003     0     0     0     0     0     0     0     1     0     0
## 10 Abrams Cre~ 2014     0     0     0     0     0     0     1     0     0     0
## 11 Abrams Cre~ 2016     0     0     0     0     0     0     1     0     0     0
## 12 Abrams Cre~ 2019     0     0     0     0     0     0     1     0     0     0
## 13 Abrams Cre~ 2022     0     0     0     0     0     0     1     0     0     0
## 14 Abrams Cre~ 1994     0     0     0     0     0     0     0     0     1     0
## 15 Abrams Cre~ 1995     0     0     0     0     0     0     0     0     1     0
## 16 Abrams Cre~ 1996     0     0     0     0     0     0     0     0     1     0
## 17 Abrams Cre~ 1997     0     0     0     0     0     0     0     0     1     0
## 18 Abrams Cre~ 1998     0     0     0     0     0     0     0     1     0     0
## 19 Abrams Cre~ 1999     0     0     0     0     0     0     0     1     0     0
## 20 Abrams Cre~ 2000     0     0     0     0     0     0     0     1     0     0
## 21 Abrams Cre~ 2001     0     0     0     0     0     0     0     1     0     0
## 22 Abrams Cre~ 2003     0     0     0     0     0     0     1     0     0     0
## 23 Abrams Cre~ 2014     0     0     0     0     0     0     1     0     0     0
## 24 Abrams Cre~ 2016     0     0     0     0     0     0     1     0     0     0
## 25 Abrams Cre~ 2019     0     0     0     0     0     0     1     0     0     0
## 26 Abrams Cre~ 2022     0     0     0     0     0     1     0     0     0     0
## 27 Alum Cave ~ 1993     0     0     0     0     0     0     1     0     0     0
## 28 Alum Cave ~ 1994     0     0     0     0     0     0     1     0     0     0
## 29 Anthony Cr~ 2014     0     0     0     0     0     1     0     0     0     0
## 30 Anthony Cr~ 2016     0     0     0     0     0     1     0     0     0     0
## 31 Anthony Cr~ 2019     0     0     0     0     0     1     0     0     0     0
## 32 Anthony Cr~ 2022     0     0     0     0     0     1     0     0     0     0

```

```
## 33 Anthony Cr~ 2014      0      0      0      0      0      1      0      0      0      0
## 34 Anthony Cr~ 2016      0      0      0      0      0      1      0      0      0      0
## 35 Anthony Cr~ 2019      0      0      0      0      0      0      1      0      0      0
## 36 Anthony Cr~ 2022      0      0      0      0      0      1      0      0      0      0
## # i 399 more rows
## # i 2 more variables: Nov <int>, Dec <int>
```

#Plot by month

```
inverts %>%
  mutate(
    Stream = stringr::word(LOC_NAME, 1, sep = ","),
    Year   = year(Start_Date)
  ) %>%
  group_by(Stream) %>%
  filter(n_distinct(Year) >= 5) %>%           # keep only 5 years sampled
  ungroup() %>%
  mutate(Month = factor(month(Start_Date, label = TRUE, abbr = TRUE),
                          levels = month.abb)) %>%
  ggplot(aes(x = Month, fill = Stream)) +
  geom_bar(position = "stack") +
  scale_fill_viridis_d(option = "turbo") +
  labs(x = "Month", y = "Number of Samples", fill = "Stream",
       title = "Overall Sampling Seasonality (5-Year Streams)") +
  theme_plot() +
  theme(panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5))
```

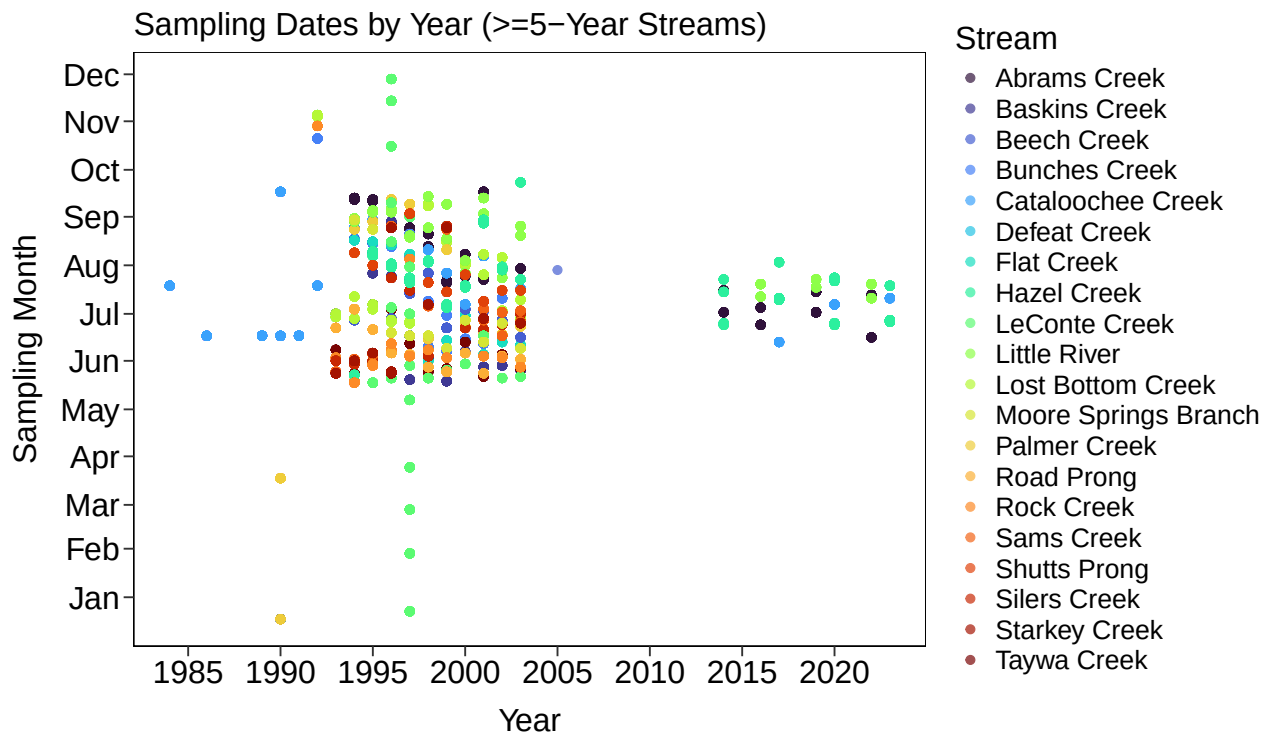


```
# scatterplot - time of year samples
inverts %>%
  mutate(
    Stream = stringr::word(LOC_NAME, 1, sep = ","),
    Year   = year(Start_Date),
```

```

    doy    = yday(Start_Date)
  ) %>%
  group_by(Stream) %>%
  filter(n_distinct(Year) >= 5) %>%
  ungroup() %>%
  ggplot(aes(x = Year, y = doy, color = Stream)) +
  geom_point(size = 2, alpha = 0.7) +
  scale_y_continuous(
    breaks = c(15, 46, 74, 105, 135, 166, 196, 227, 258, 288, 319, 349),
    labels = month.abb
  ) +
  scale_x_continuous(breaks = seq(1980, 2025, by = 5)) +
  scale_color_viridis_d(option = "turbo") +
  labs(x = "Year", y = "Sampling Month", color = "Stream",
       title = "Sampling Dates by Year (5-Year Streams)") +
  theme_plot() +
  theme(panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5))

```



```

# scatterplot - time of year samples
inverts %>%
  mutate(
    Stream = stringr::word(LOC_NAME, 1, sep = ","),
    Year   = year(Start_Date),
    doy    = yday(Start_Date)
  ) %>%
  group_by(Stream) %>%
  filter(n_distinct(Year) >= 5) %>%
  ungroup() %>%
  ggplot(aes(y = Year, x = doy, color = Stream)) +
  geom_point(size = 2, alpha = 0.7) +

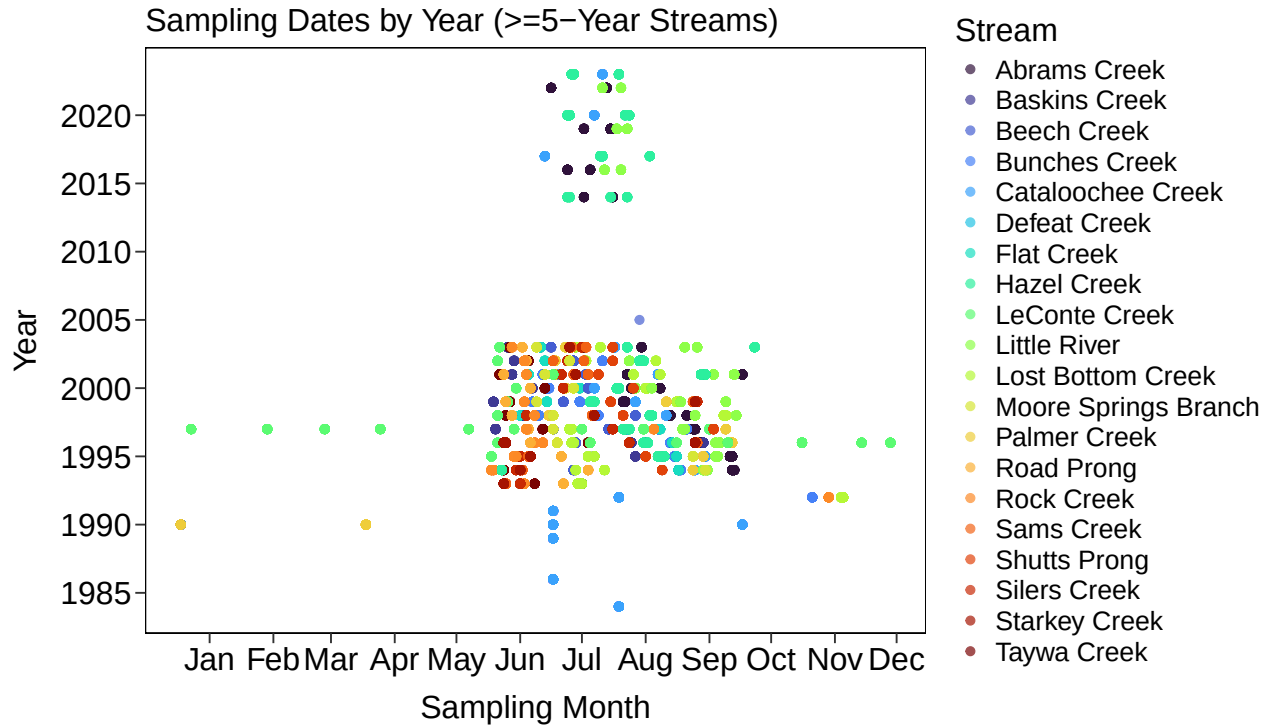
```



```

scale_x_continuous(
  breaks = c(15, 46, 74, 105, 135, 166, 196, 227, 258, 288, 319, 349),
  labels = month.abb
) +
scale_y_continuous(breaks = seq(1980, 2025, by = 5)) +
scale_color_viridis_d(option = "turbo") +
labs(x = "Sampling Month", y = "Year", color = "Stream",
     title = "Sampling Dates by Year (5-Year Streams)") +
theme_plot() +
theme(panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5))

```



```

# Multiple samples?
# Count number of distinct sample codes per site-year
multisite_samples <- invert %>%
  group_by(LOC_NAME, Year) %>%
  summarise(
    n_samplecodes = n_distinct(Sample_Code),
    n_days = n_distinct(Start_Date),
    sample_codes = paste(unique(Sample_Code), collapse = ", "),
    .groups = "drop"
  ) %>%
  filter(n_samplecodes > 1)

#
multisite_samples %>% print(n = 11)

## # A tibble: 13 x 5
##   LOC_NAME                Year n_samplecodes n_days sample_codes
##   <chr>                  <dbl>         <int>   <int> <chr>
## 1 Bear Creek, Site 1      1994             2         2 FOBC011994b, FOBC0119~
## 2 Cataloochee Creek, Site 1 1989             2         1 CTCC011989a, CTCC0119~

```

```
## 3 Cataloochee Creek, Site 1      1990      4      3 CTCC011990a, CTCC0119~
## 4 LeConte Creek, Site 1         1996      7      7 LCLC011996a, LCLC0119~
## 5 LeConte Creek, Site 1         1997      9      9 LCLC011997a, LCLC0119~
## 6 Lower Double Branch, Site 1   1990      4      4 CTLD011990a, CTLD0119~
## 7 Palmer Creek, Site 2          1990      2      2 CTPC021990a, CTPC0219~
## 8 Sams Creek, Site 1            1996      2      2 UTSA71996a, UTSA71996b
## 9 Starkey Creek, Site 1         1996      2      2 UTST41996a, UTST41996b
## 10 Starkey Creek, Site 2        1996      4      3 UTST31996b, UTST51996~
## 11 Thunderhead Prong, Site 2    1996      2      2 UTSA81996a, UTSA81996b
## # i 2 more rows
```

```
#---- Average in cases of multiple samples per site
```

```
# first average for a given sample code (usually only one but not always)
```

```
EPT_ORDERS <- c("Ephemeroptera","Plecoptera","Trichoptera")
```

```
sample_level <- invert %>%
```

```
  group_by(LOC_NAME, Year, Start_Date, Sample_Code) %>%
```

```
  summarise(
```

```
    abund_sample = sum(Count, na.rm = TRUE),
```

```
    rich_sample   = n_distinct(Genus, na.rm = TRUE),
```

```
    EPT_rich_sample = n_distinct(Genus[Lab_Order %in% c("Ephemeroptera","Plecoptera","Trichoptera")], na.rm = TRUE),
```

```
    EPT_abund_sample = sum(Count[Lab_Order %in% EPT_ORDERS], na.rm = TRUE),
```

```
    ncbi_sample = sum(genus_ncbi * Count, na.rm = TRUE)/sum(Count, na.rm = TRUE),
```

```
    .groups = "drop"
```

```
)
```

```
# then average if multiple samples per day
```

```
site_day <- sample_level %>%
```

```
  group_by(LOC_NAME, Year, Start_Date) %>%
```

```
  summarise(
```

```
    abund_day = mean(abund_sample, na.rm = TRUE),
```

```
    rich_day   = mean(rich_sample, na.rm = TRUE),
```

```
    EPT_rich_day = mean(EPT_rich_sample, na.rm = TRUE),
```

```
    EPT_abund_day = mean(EPT_abund_sample, na.rm = TRUE),
```

```
    ncbi_day = mean(ncbi_sample, na.rm = TRUE),
```

```
    n_samples = n(), # QC: how many samples contributed
```

```
    .groups = "drop"
```

```
)
```

```
# Average across days per site
```

```
site_year <- site_day %>%
```

```
  group_by(LOC_NAME, Year) %>%
```

```
  summarise(
```

```
    abund_site = mean(abund_day, na.rm = TRUE),
```

```
    rich_site   = mean(rich_day, na.rm = TRUE),
```

```
    EPT_rich_site = mean(EPT_rich_day, na.rm = TRUE),
```

```
    EPT_abund_site = mean(EPT_abund_day, na.rm = TRUE),
```

```
    ncbi_site = mean(ncbi_day, na.rm = TRUE),
```

```
    n_samples = n(), # QC: how many samples contributed
```

```
    .groups = "drop"
```

```
)
```

```
# average across sites per stream
```

```

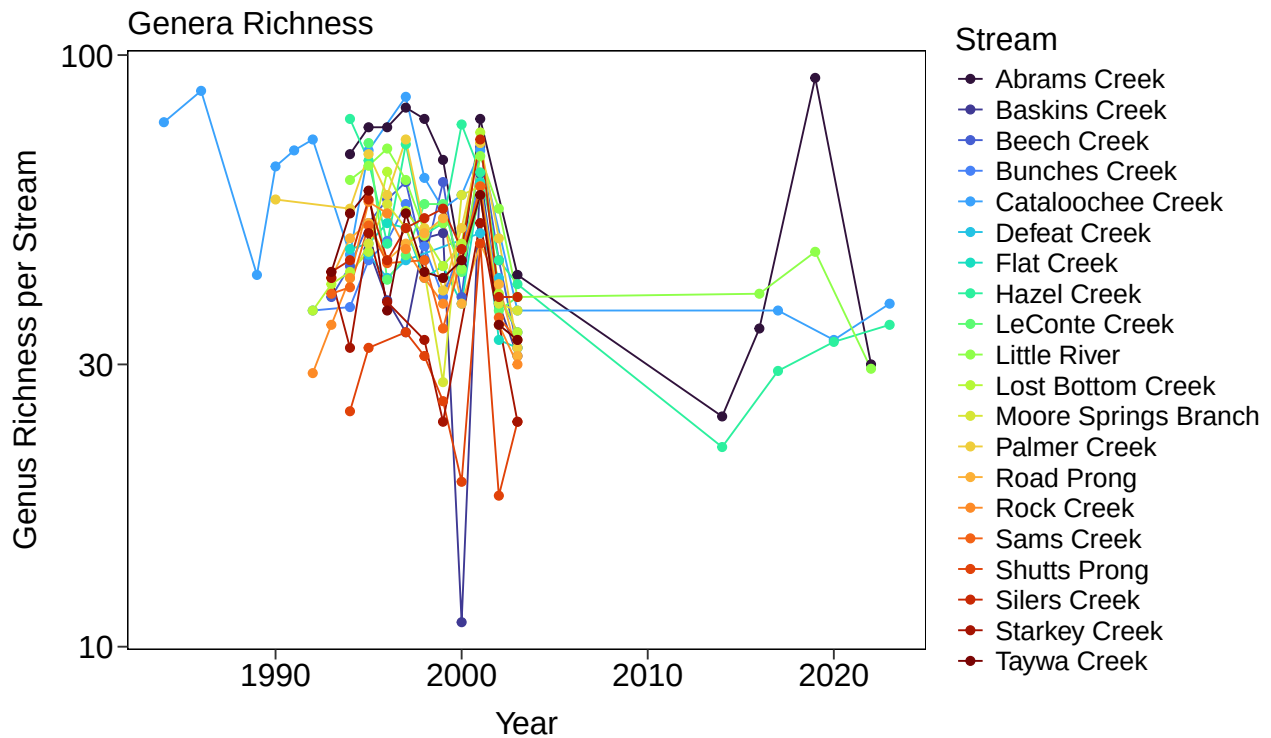
inverts_stream <- site_year %>%
  mutate(Stream = word(LOC_NAME, 1, sep = ",")) %>%
  group_by(Stream, Year) %>%
  summarise(
    abund_stream = mean(abund_site, na.rm = TRUE),
    rich_stream = mean(rich_site, na.rm = TRUE),
    EPT_rich_stream = mean(EPT_rich_site, na.rm = TRUE),
    EPT_abund_stream = mean(EPT_abund_site, na.rm = TRUE),
    ncbi_stream = mean(ncbi_site, na.rm = TRUE),
    EPT_prop_abun = EPT_abund_stream / abund_stream,
    n_samples = n(), # QC: how many samples contributed
    .groups = "drop"
  ) %>%
  group_by(Stream) %>%
  mutate(n_years_sampled = n_distinct(Year)) %>% # total years that stream was sampled
  ungroup()

# check

# ----- Plots -----

# --- Richness
p_richness <- ggplot(
  invert_stream %>% filter(n_years_sampled >= 5, abund_stream > 0),
  aes(x = Year, y = rich_stream, color = Stream, group = Stream)) +
  geom_line() +
  geom_point(size = 2) +
  #geom_smooth(method = "lm", linetype = "dashed", se = F, size = .75) +
  scale_y_log10() +
  scale_x_continuous(breaks = seq(1980, 2020, by = 10)) +
  labs(x = "Year", y = "Genus Richness per Stream", color = "Stream") +
  theme_plot() +
  ggtitle("Genera Richness") +
  scale_color_viridis_d(option = "turbo") +
  theme(
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    legend.position = "right")
p_richness

```

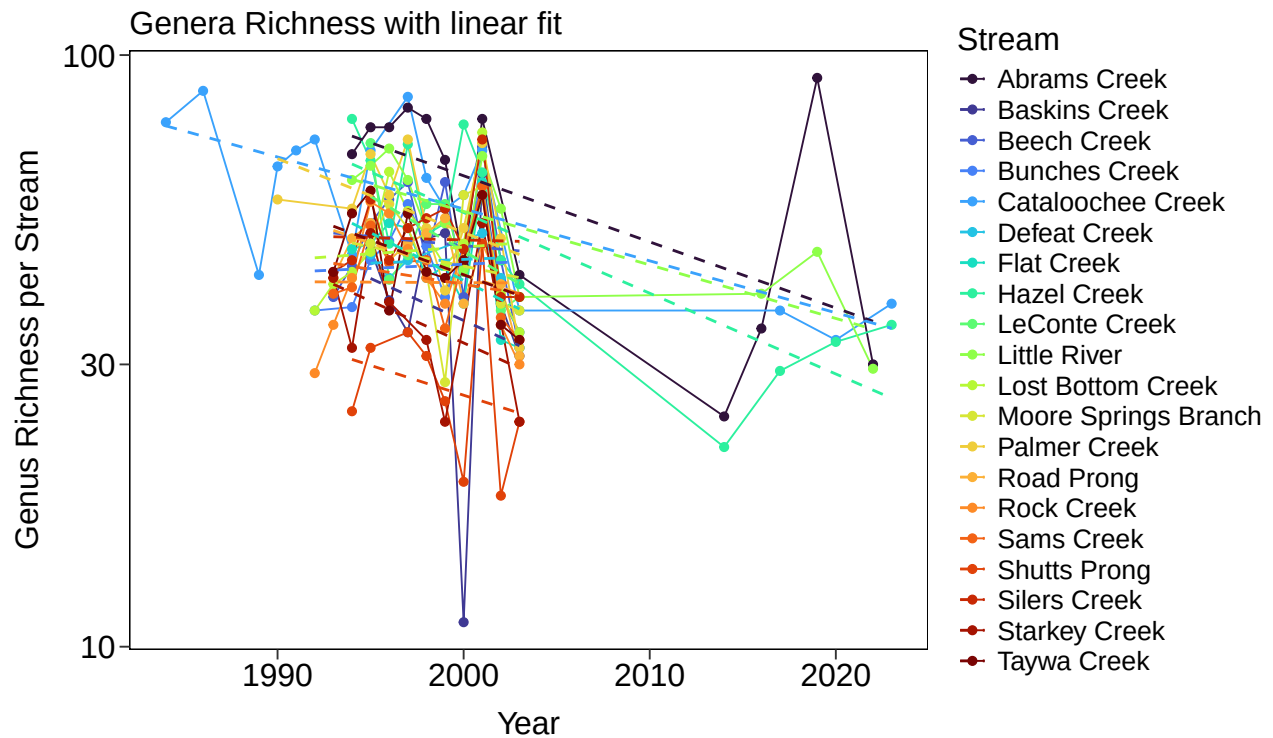


```
# Add linear fit
p_richness <- ggplot(
  invert_stream %>% filter(n_years_sampled >= 5, abund_stream > 0),
  aes(x = Year, y = rich_stream, color = Stream, group = Stream)) +
  geom_line() +
  geom_point(size = 2) +
  geom_smooth(method = "lm", linetype = "dashed", se = F, size = .75) +
  scale_y_log10() +
  scale_x_continuous(breaks = seq(1980, 2025, by = 10)) +
  labs(x = "Year", y = "Genus Richness per Stream", color = "Stream") +
  theme_plot() +
  ggtitle("Genera Richness with linear fit") +
  scale_color_viridis_d(option = "turbo") +
  theme(
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    legend.position = "right")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
p_richness
```

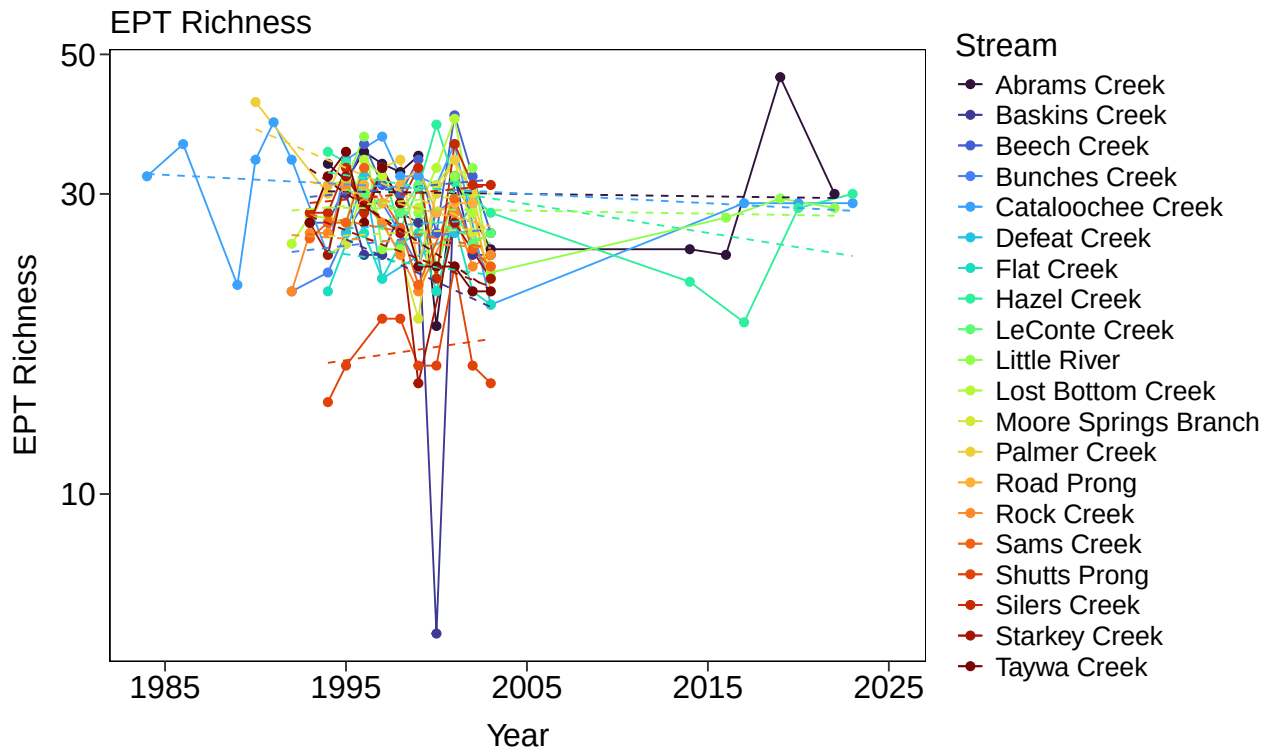
```
## `geom_smooth()` using formula = 'y ~ x'
```



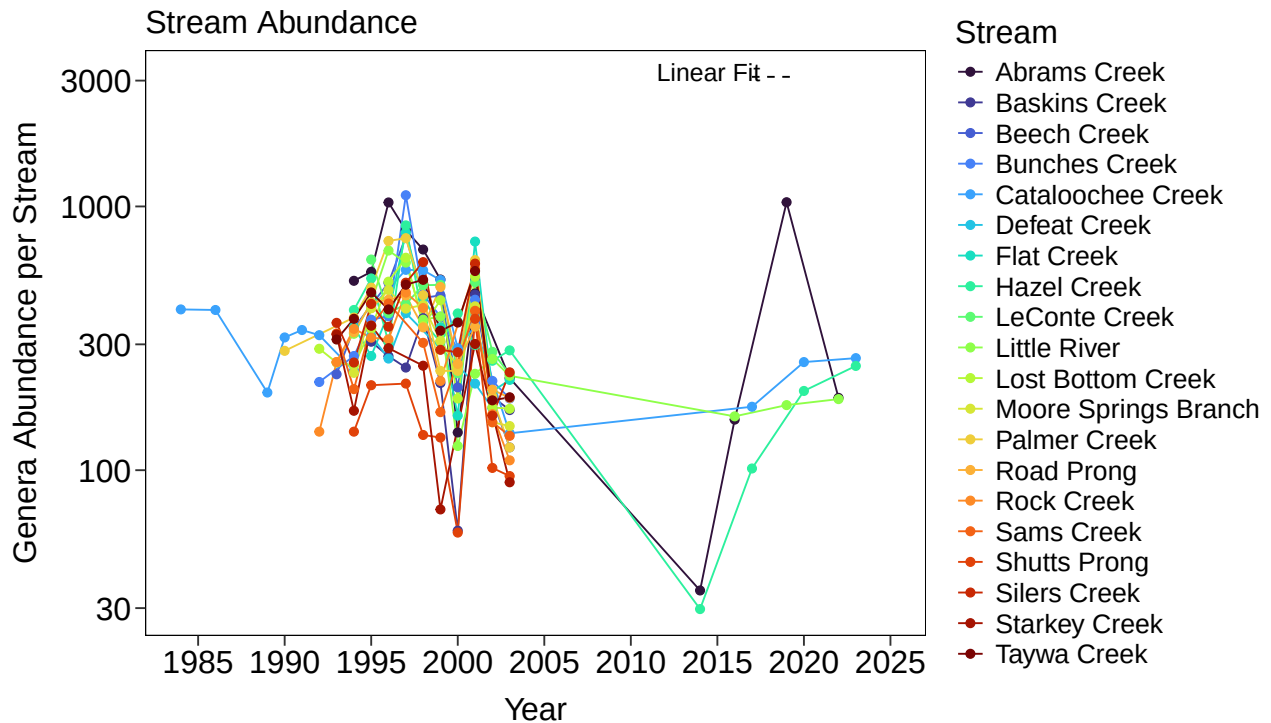
```
# EPT Richness

p_ept <- ggplot(
  invert_stream %>%
    filter(n_years_sampled >= 5, abund_stream > 0),
  aes(x = Year, y = EPT_rich_stream, color = Stream, group = Stream)) +
  geom_line() +
  geom_point(size = 2) +
  geom_smooth(method = "lm", linetype = "dashed", se = F, size = .5) +
  scale_x_continuous(
    breaks = seq(1985, 2025, by = 10),
    limits = c(1984, 2025)) +
  labs(x = "Year", y = "EPT", color = "Stream") +
  scale_y_log10("EPT Richness") +
  theme_plot() +
  ggtitle("EPT Richness") +
  scale_color_viridis_d(option = "turbo") +
  theme(
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    legend.position = "right")
p_ept

## `geom_smooth()` using formula = 'y ~ x'
```

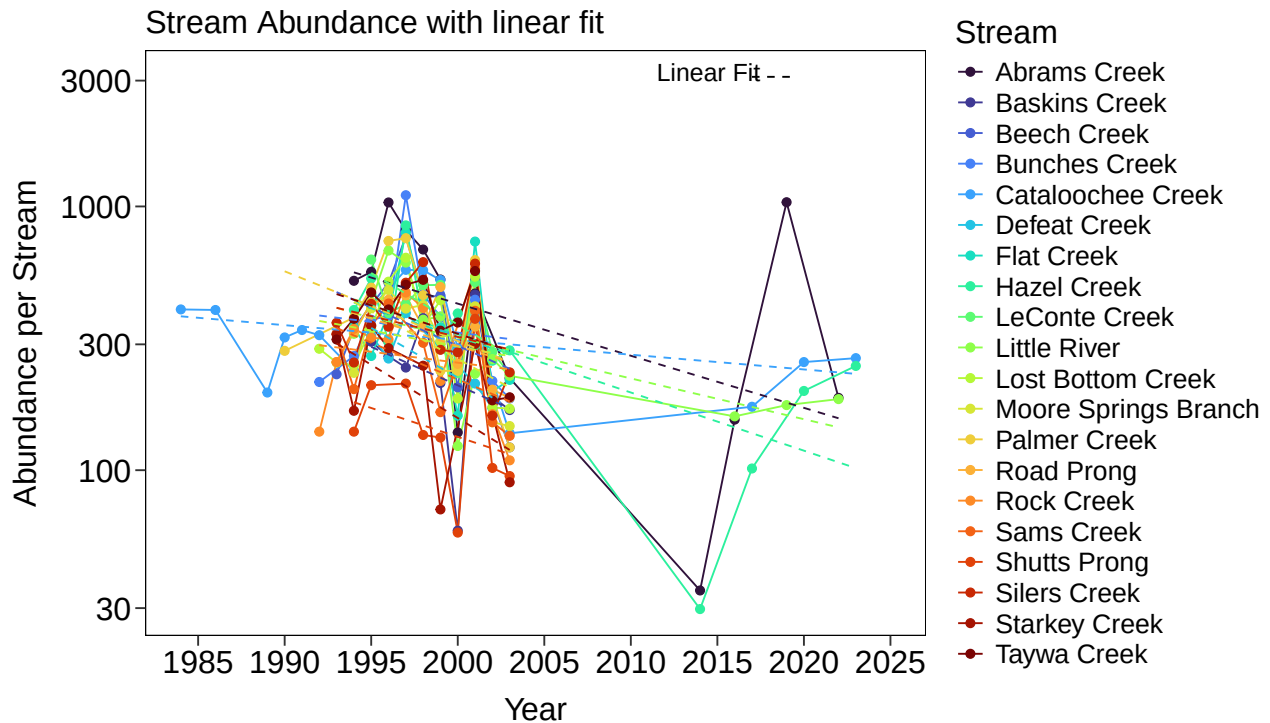


```
# --- Abundance
p_abundance <- ggplot(
  invert_stream %>%
    filter(n_years_sampled >= 5, abund_stream > 0),
  aes(x = Year, y = abund_stream, color = Stream, group = Stream)) +
  geom_line() +
  geom_point(size = 2) +
  scale_y_log10() +
  scale_x_continuous(
    breaks = seq(1985, 2025, by = 5),
    limits = c(1984, 2025)) +
  labs(x = "Year", y = "Genera Abundance per Stream", color = "Stream") +
  theme_plot() +
  ggtitle("Stream Abundance") +
  scale_color_viridis_d(option = "turbo") +
  theme( panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    legend.position = "right") +
  annotate("segment", x = 2017, xend = 2019.5, y = 3100, yend = 3100,
    color = "black", linetype = "dashed") +
  annotate("text", x = 2011.5, y = 3000, label = "Linear Fit", size = 5,
    hjust = 0, vjust = 0, color = "black")
p_abundance
```



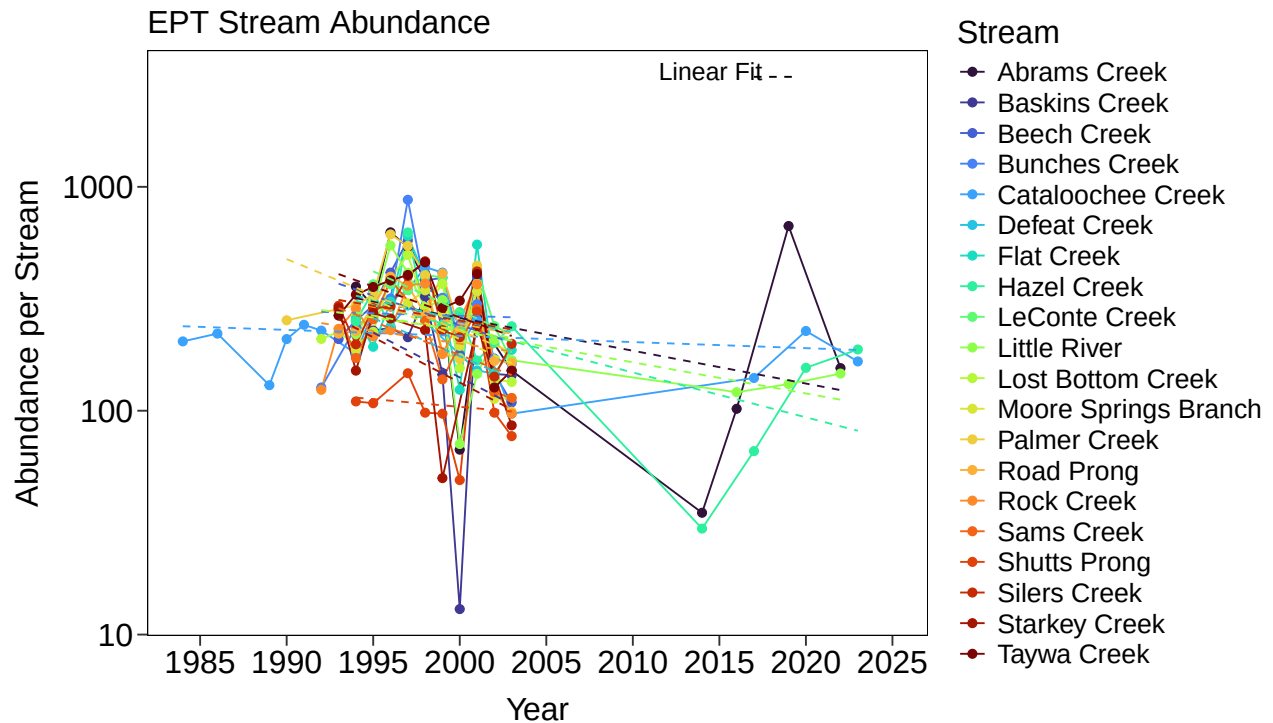
```
# with linear fit
p_abundance <- ggplot(
  invert_stream %>%
    filter(n_years_sampled >= 5, abund_stream > 0),
  aes(x = Year, y = abund_stream, color = Stream, group = Stream)) +
  geom_line() +
  geom_point(size = 2) +
  stat_smooth(method = "lm", linetype = "dashed", se = F, show.legend = F, size = 0.5) +
  scale_y_log10() +
  scale_x_continuous(
    breaks = seq(1985, 2025, by = 5),
    limits = c(1984, 2025)) +
  labs(x = "Year", y = "Abundance per Stream", color = "Stream") +
  theme_plot() +
  ggtitle("Stream Abundance with linear fit") +
  scale_color_viridis_d(option = "turbo") +
  theme( panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    legend.position = "right") +
  annotate("segment", x = 2017, xend = 2019.5, y = 3100, yend = 3100,
    color = "black", linetype = "dashed") +
  annotate("text", x = 2011.5, y = 3000, label = "Linear Fit", size = 5,
    hjust = 0, vjust = 0, color = "black")
p_abundance

## `geom_smooth()` using formula = 'y ~ x'
```



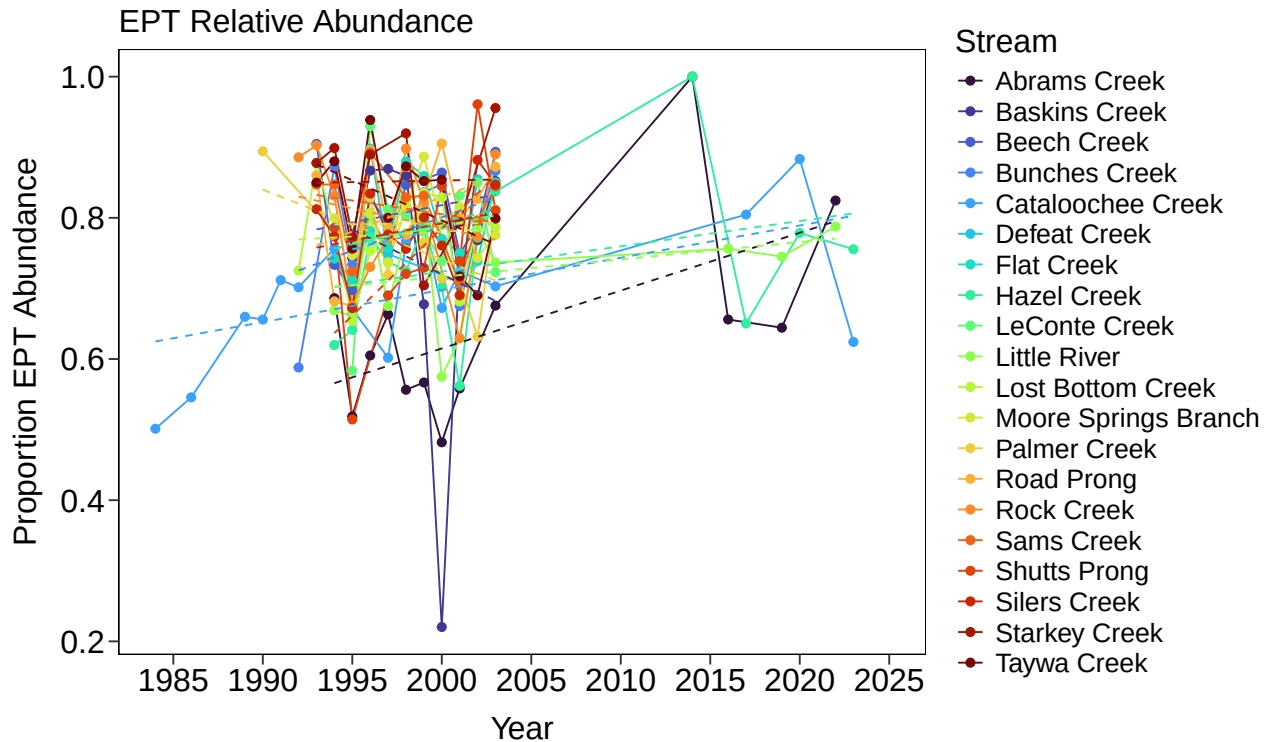
```
# EPT abundance
p_ept_abundance <- ggplot(
  invert_stream %>%
    filter(n_years_sampled >= 5, abund_stream > 0),
  aes(x = Year, y = EPT_abund_stream, color = Stream, group = Stream)) +
  geom_line() +
  geom_point(size = 2) +
  stat_smooth(method = "lm", linetype = "dashed", se = F, show.legend = F, size = 0.5) +
  scale_y_log10() +
  scale_x_continuous(
    breaks = seq(1985, 2025, by = 5),
    limits = c(1984, 2025)) +
  labs(x = "Year", y = "Abundance per Stream", color = "Stream") +
  theme_plot() +
  ggtitle("EPT Stream Abundance") +
  scale_color_viridis_d(option = "turbo") +
  theme( panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    legend.position = "right") +
  annotate("segment", x = 2017, xend = 2019.5, y = 3100, yend = 3100,
    color = "black", linetype = "dashed") +
  annotate("text", x = 2011.5, y = 3000, label = "Linear Fit", size = 5,
    hjust = 0, vjust = 0, color = "black")
p_ept_abundance

## `geom_smooth()` using formula = 'y ~ x'
```

```
# Relative EPT abundance
p_rel_ept <- ggplot(
  invert_stream %>%
    filter(n_years_sampled >= 5, abund_stream > 0),
  aes(x = Year, y = EPT_prop_abun, color = Stream, group = Stream)) +
  geom_line() +
  geom_point(size = 2) +
  stat_smooth(method = "lm", linetype = "dashed", se = F, show.legend = F, size = 0.5) +
  scale_y_continuous("Proportion EPT Abundance") +
  scale_x_continuous(
    breaks = seq(1985, 2025, by = 5),
    limits = c(1984, 2025)) +
  labs(x = "Year", y = "Abundance per Stream", color = "Stream") +
  theme_plot() +
  ggtitle("EPT Relative Abundance") +
  scale_color_viridis_d(option = "turbo") +
  theme( panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    legend.position = "right")
p_rel_ept
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
# --- NCBI
ref_lines <- data.frame(
  y = c(4.18, 5.09, 5.91, 7.05),
  label = c("Excellent", "Good", "Good-Fair", "Poor"))
x_left <- min(inverts_stream$Year, na.rm = TRUE)

p_ncbi <- ggplot(
  inverts_stream %>%
    filter(n_years_sampled >= 5, abund_stream > 0),
  aes(x = Year, y = ncbi_stream, color = Stream, group = Stream)) +
  geom_line() +
  geom_point(size = 2) +
  geom_smooth(method = "lm", linetype = "dashed", se = F, size = .75) +
  scale_x_continuous(
    breaks = seq(1985, 2025, by = 5),
    limits = c(1984, 2025)) +
  labs(x = "Year", y = "NCBI", color = "Stream") +
  scale_y_continuous(name = "NCBI per Stream", limits = c(0.5, 4.4)) +
  geom_text(
    data = ref_lines, aes(x = x_left, y = y + 0.1, label = label),
    color = "grey30", size = 4, hjust = 0, inherit.aes = FALSE) +
  theme_plot() +
  scale_color_viridis_d(option = "turbo") +
  theme(panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    legend.position = "right") +
  ggtitle("Stream NCBI") +
  annotate("segment", x = 2017, xend = 2019.5, y = 3.9, yend = 3.9,
    color = "black", linetype = "dashed") +
```

```

annotate("text", x = 2011.5, y = 3.85, label = "Linear Fit", size = 5,
        hjust = 0, vjust = 0, color = "black") +
geom_hline(data = ref_lines, aes(yintercept = y), color = "grey40",
          linetype = "dashed", linewidth = 0.8, inherit.aes = FALSE)

```

```

## Warning in geom_hline(data = ref_lines, aes(yintercept = y), color = "grey40",
## : Ignoring unknown parameters: `inherit.aes`

```

```
p_ncbi
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```

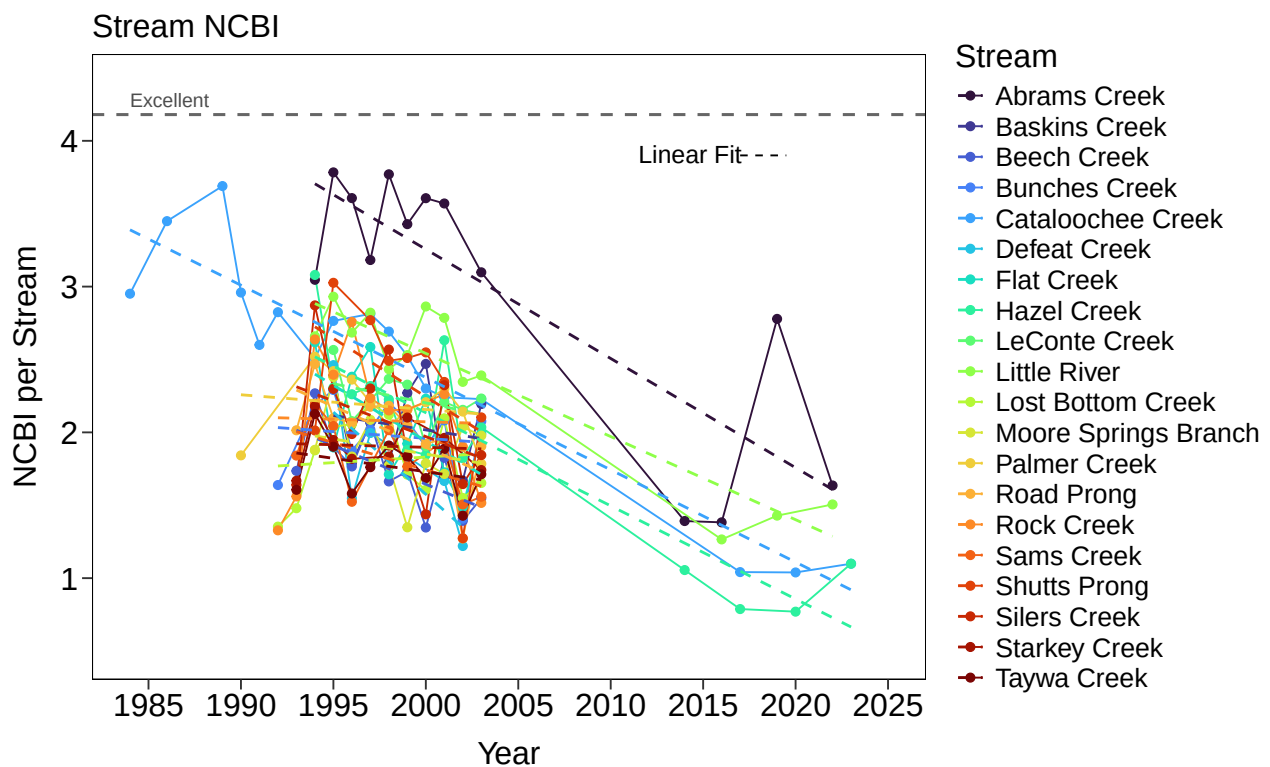
## Warning: Removed 3 rows containing missing values or values outside the scale range
## (`geom_text()`).

```

```

## Warning: Removed 3 rows containing missing values or values outside the scale range
## (`geom_hline()`).

```



Month by month comparison over the summer

```

# -----
# 1) Build month-aware summaries
# -----

# Add Month once (ordered Jan..Dec)
inverts_m <- invert %>%
  mutate(
    Month = factor(month(Start_Date), levels = 1:12, labels = month.abb),
    Year = year(Start_Date)
  )

EPT_ORDERS <- c("Ephemeroptera", "Plecoptera", "Trichoptera")

```

```

# sample level (unique Sample_Code within day)
sample_level_m <- invert_m %>%
  group_by(LOC_NAME, Year, Month, Start_Date, Sample_Code) %>%
  summarise(
    abund_sample = sum(Count, na.rm = TRUE),
    rich_sample = n_distinct(Genus, na.rm = TRUE),
    EPT_rich_sample = n_distinct(Genus[Lab_Order %in% EPT_ORDERS], na.rm = TRUE),
    EPT_abund_sample = sum(Count[Lab_Order %in% EPT_ORDERS], na.rm = TRUE),
    ncbi_sample = sum(genus_ncbi * Count, na.rm = TRUE) / sum(Count, na.rm = TRUE),
    .groups = "drop"
  )

# day level
site_day_m <- sample_level_m %>%
  group_by(LOC_NAME, Year, Month, Start_Date) %>%
  summarise(
    abund_day = mean(abund_sample, na.rm = TRUE),
    rich_day = mean(rich_sample, na.rm = TRUE),
    EPT_rich_day = mean(EPT_rich_sample, na.rm = TRUE),
    EPT_abund_day = mean(EPT_abund_sample, na.rm = TRUE),
    ncbi_day = mean(ncbi_sample, na.rm = TRUE),
    n_samples = n(),
    .groups = "drop"
  )

# site × year × month
site_year_month <- site_day_m %>%
  group_by(LOC_NAME, Year, Month) %>%
  summarise(
    abund_site = mean(abund_day, na.rm = TRUE),
    rich_site = mean(rich_day, na.rm = TRUE),
    EPT_rich_site = mean(EPT_rich_day, na.rm = TRUE),
    EPT_abund_site = mean(EPT_abund_day, na.rm = TRUE),
    ncbi_site = mean(ncbi_day, na.rm = TRUE),
    n_days = n(),
    .groups = "drop"
  )

# stream × year × month
invert_stream_m <- site_year_month %>%
  mutate(Stream = stringr::word(LOC_NAME, 1, sep = ",")) %>%
  group_by(Stream, Year, Month) %>%
  summarise(
    abund_stream = mean(abund_site, na.rm = TRUE),
    rich_stream = mean(rich_site, na.rm = TRUE),
    EPT_rich_stream = mean(EPT_rich_site, na.rm = TRUE),
    EPT_abund_stream = mean(EPT_abund_site, na.rm = TRUE),
    ncbi_stream = mean(ncbi_site, na.rm = TRUE),
    EPT_prop_abun = if_else(abund_stream > 0, EPT_abund_stream / abund_stream, NA_real_),
    n_sites_agg = n(), # how many sites contributed in that month
    .groups = "drop"
  ) %>%
  group_by(Stream, Month) %>%

```

```

mutate(n_years_sampled = dplyr::n_distinct(Year)) %>% # total years sampled for this Stream in THIS
ungroup()

# --- Helper function: single wide summer plot ---
summer_plot <- function(df, yvar, ylab, title, logy = FALSE, ylim = NULL) {

  df_summer <- df %>%
    filter(Month %in% c("Jun", "Jul", "Aug")) %>%
    group_by(Stream, Month) %>%
    mutate(n_years_sampled = dplyr::n_distinct(Year)) %>%
    ungroup() %>%
    filter(n_years_sampled >= 5) %>%
    mutate(Month = forcats::fct_relevel(Month, "Jun", "Jul", "Aug"))

  if (isTRUE(logy)) {
    df_summer <- df_summer %>% filter(.data[[yvar]] > 0)
  }

  p <- ggplot(
    df_summer,
    aes(x = Year, y = .data[[yvar]], color = Stream, group = Stream)
  ) +
    geom_line() +
    geom_point(size = 2) +
    geom_smooth(method = "lm", linetype = "dashed", se = FALSE, size = 0.7) +
    scale_x_continuous(breaks = seq(1980, 2025, by = 10), limits = c(1984, 2025)) +
    labs(x = "Year", y = ylab, color = "Stream", title = title) +
    theme_plot() +
    scale_color_viridis_d(option = "turbo") +
    facet_wrap(~ Month, ncol = 1, strip.position = "left",
               labeller = as_labeller(c(Jun = "June", Jul = "July", Aug = "August")))) +
    theme(
      strip.text      = element_text(size = 18, face = "bold"),
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      panel.border     = element_rect(color = "black", fill = NA, linewidth = 0.5),
      legend.position  = "right"
    )

  if (isTRUE(logy)) {
    p <- p + scale_y_log10()
  } else if (!is.null(ylim)) {
    p <- p + scale_y_continuous(limits = ylim)
  }

  p + theme(plot.margin = margin(10, 20, 10, 10),
            aspect.ratio = 0.25)
}

# --- Define metrics (one plot each) ---
metrics <- tribble(
  ~yvar,          ~ylab,          ~title,          ~logy, ~ylim,
  "rich_stream",  "Genus Richness per Stream",  "Genera Richness - Summer", TRUE,  NA,

```

```

"abund_stream",      "Abundance per Stream",      "Stream Abundance - Summer",    TRUE,    NA,
"EPT_rich_stream",   "EPT Richness per Stream",    "EPT Richness - Summer",       TRUE,    NA,
"EPT_prop_abun",     "EPT Relative Abundance",    "EPT Relative Abundance - Summer", FALSE,    c(0, 1),
"ncbi_stream",       "NCBI per Stream",           "NCBI - Summer",               FALSE,    c(0.5, 1)
)

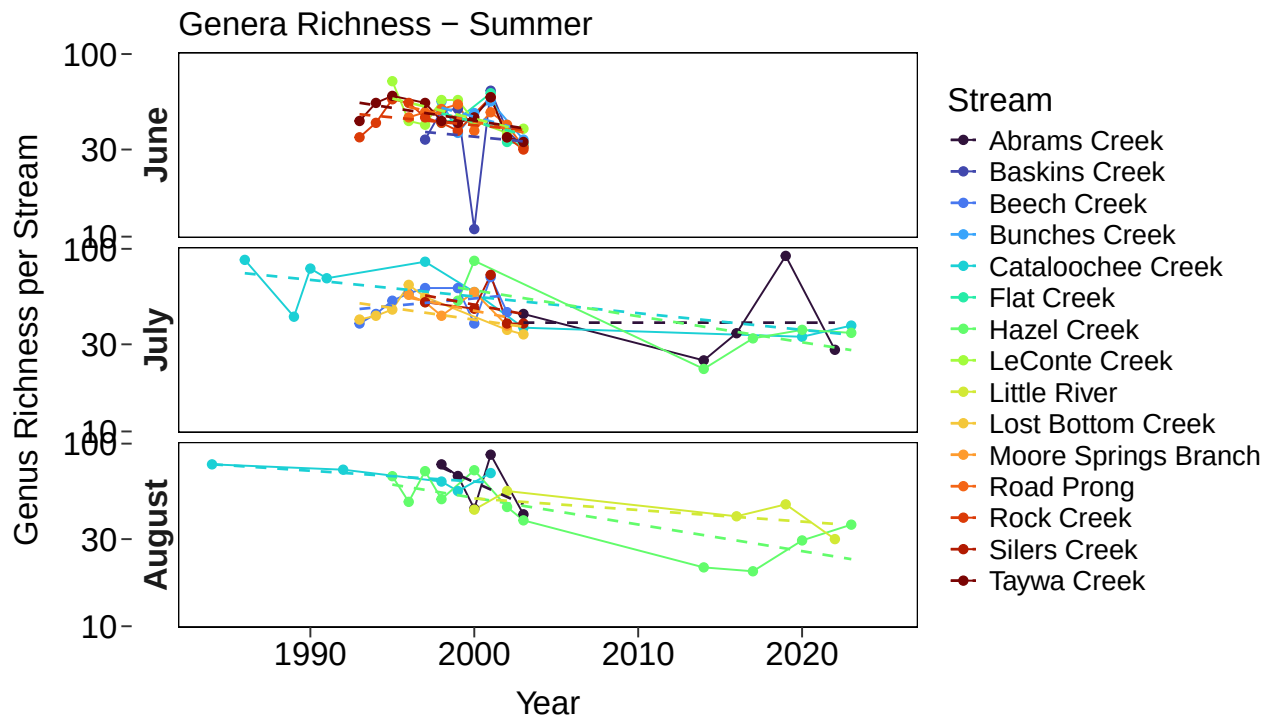
# --- Generate all plots ---
summer_plots <- metrics %>%
  mutate(plot = pmap(list(yvar, ylab, title, logy, ylim),
    ~ summer_plot(inverts_stream_m, ..1, ..2, ..3, ..4, ..5)))

# --- Access individual plots ---
p_richness_summer <- summer_plots$plot[[1]]
p_abundance_summer <- summer_plots$plot[[2]]
p_ept_rich_summer <- summer_plots$plot[[3]]
p_ept_prop_summer <- summer_plots$plot[[4]]
p_ncbi_summer <- summer_plots$plot[[5]]

# --- Display any plot ---
p_richness_summer

## `geom_smooth()` using formula = 'y ~ x'

```

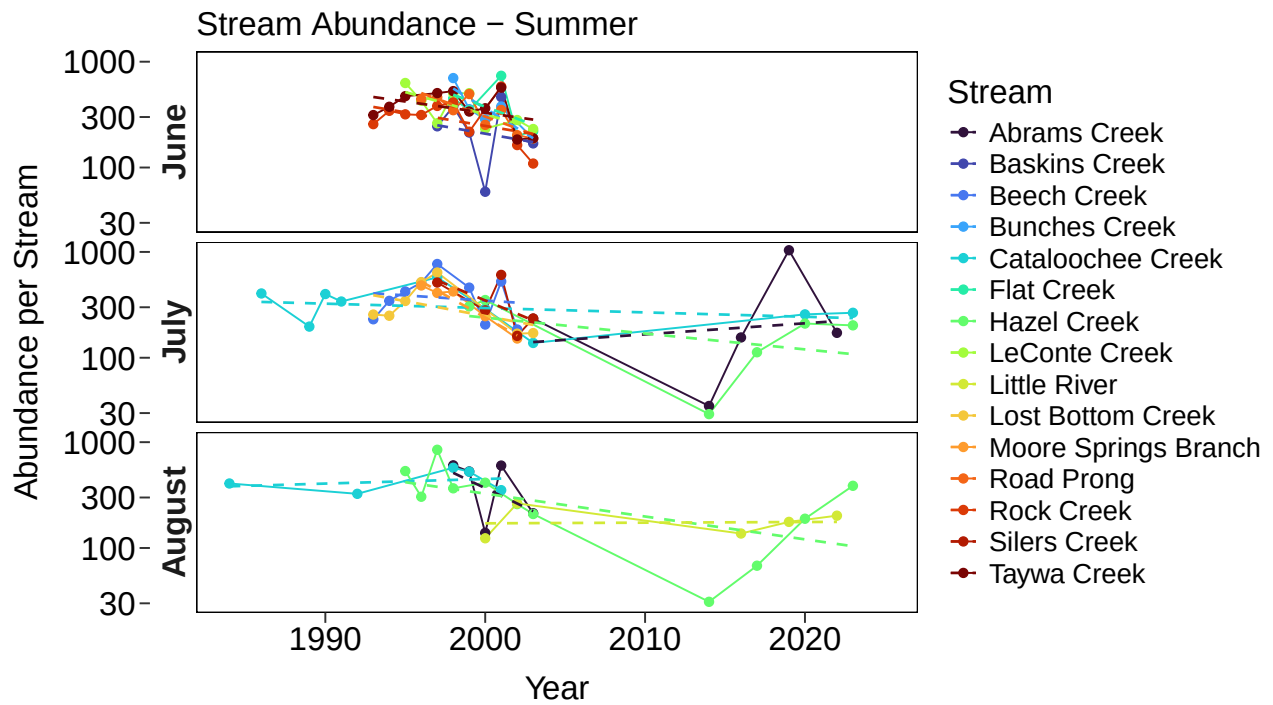


```

p_abundance_summer

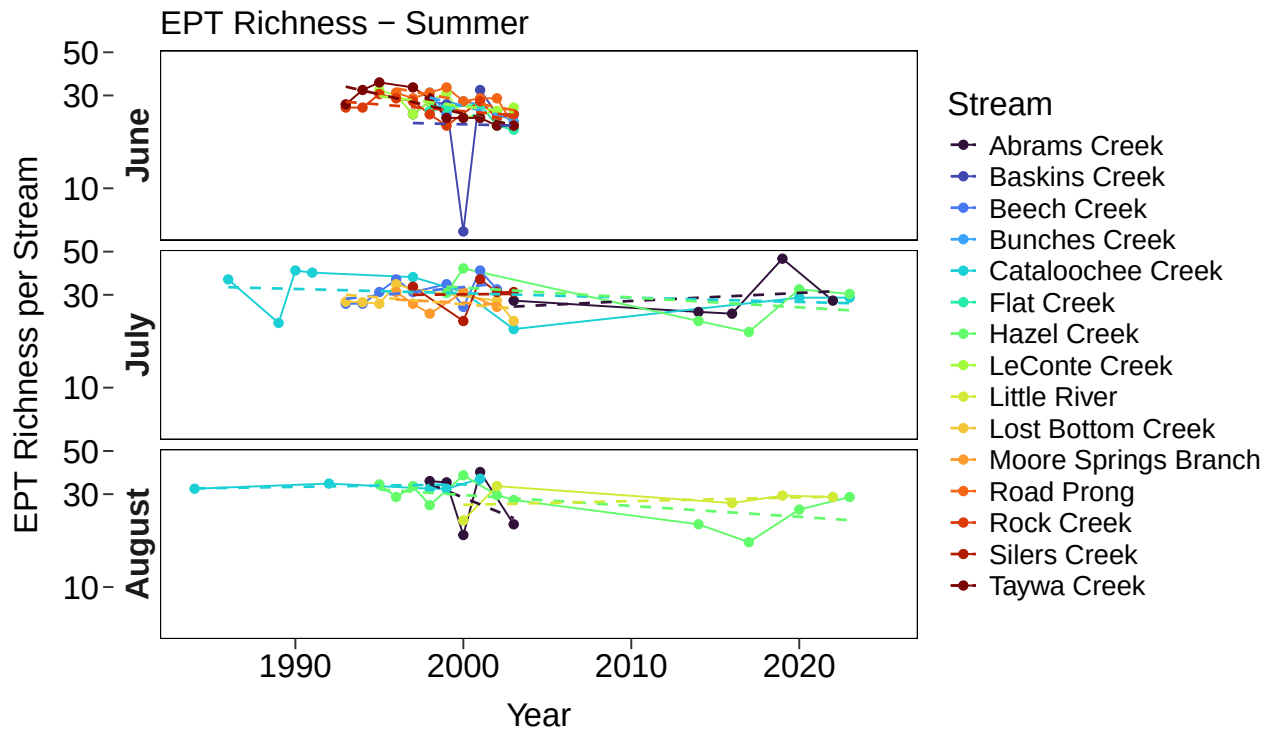
## `geom_smooth()` using formula = 'y ~ x'

```



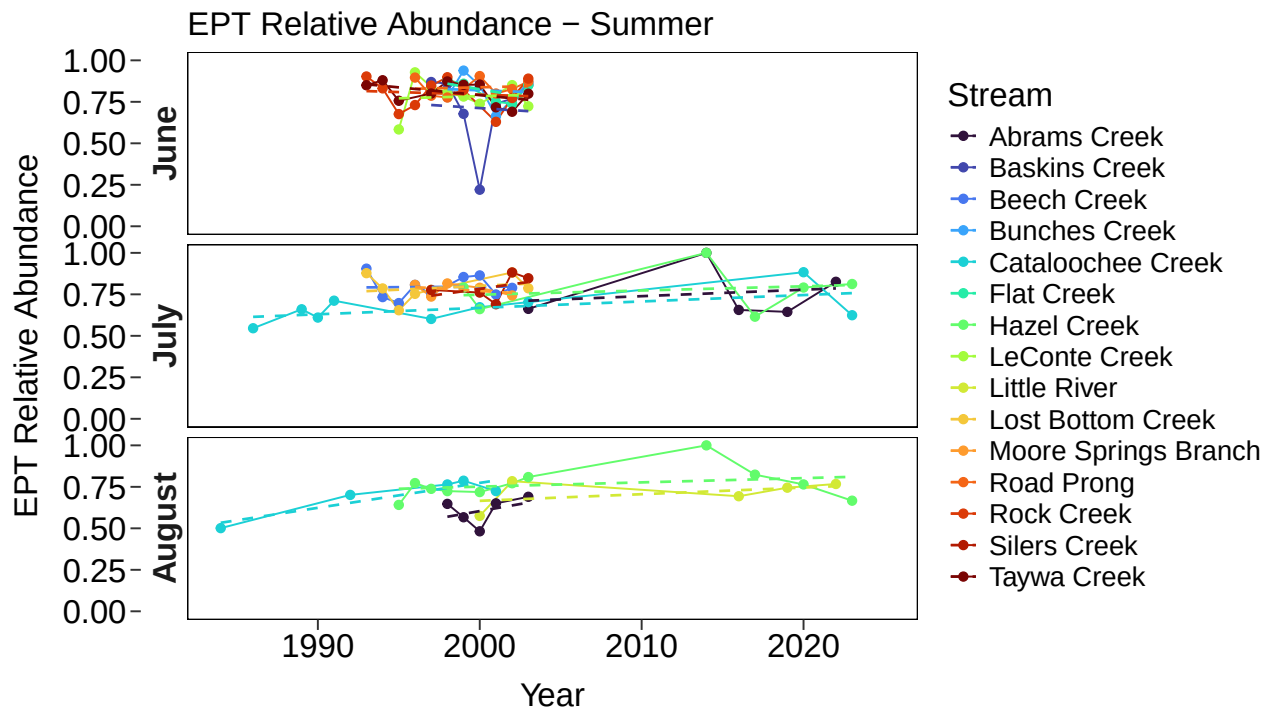
```
p_ept_rich_summer
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



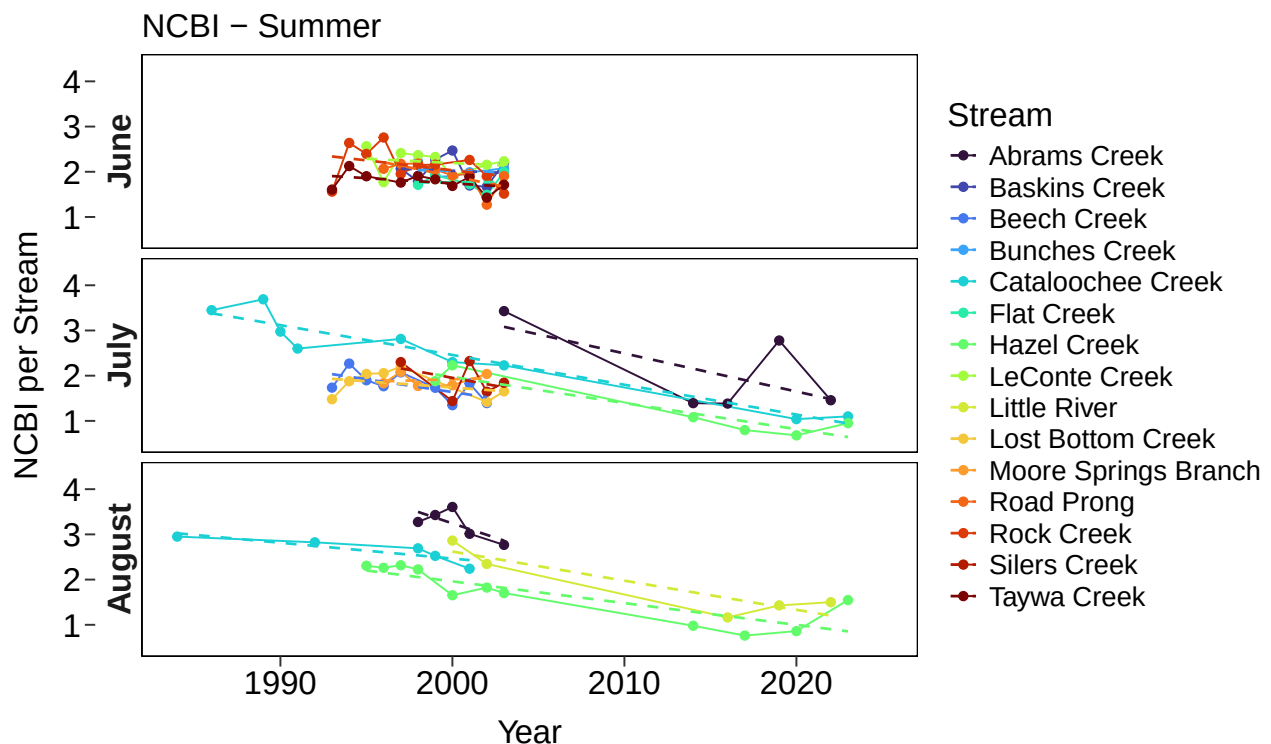
```
p_ept_prop_summer
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
p_ncbi_summer
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```
#----- Attach geometry for plotting
```

```
coords_sf <- invert %>%
```

```
st_as_sf(coords = c("LON", "LAT"), crs = 4326, remove = FALSE) %>%
```



```

mutate(stream = str_squish(str_remove(LOC_NAME, ",?\\s*Site\\s*\\w+$"))) %>%
group_by(stream) %>%
select(-Year) %>%
slice_head(n = 1) # one actual sampled point per stream

inverts_stream_sf <- invert_stream %>%
  left_join(coords_sf %>% dplyr::rename(Stream = stream), by = "Stream") %>%
  sf::st_as_sf()

inverts_stream_sf <- st_transform(inverts_stream_sf, st_crs(grsm_watershed))

# Minimal prep for colorizing by name
streams_named <- grsm_streams %>%
  filter(!is.na(gnis_name), gnis_name != "") %>%
  mutate(color_id = factor(gnis_name)) %>%
  rename(Stream = gnis_name)

loc_used <- invert_stream_sf %>%
  st_drop_geometry() %>%
  dplyr::distinct(Stream)
streams_used <- streams_named %>%
  dplyr::semi_join(loc_used, by = "Stream")

# join locations to coords; keep one point per Location
sites_pts <- coords_sf %>%
  inner_join(loc_used, by = c("stream" = "Stream")) %>% # keep only sampled sites
  dplyr::select(stream, geometry) %>%
  sf::st_transform(sf::st_crs(grsm_watershed))

# one label point per stream name (keep all stream lines)
stream_labels <- streams_used %>%
  dplyr::group_by(Stream, color_id) %>%
  dplyr::summarise(do_union = TRUE, .groups = "drop") %>%
  sf::st_transform(26917) %>%
  dplyr::mutate(geometry = sf::st_point_on_surface(geom)) %>% # one point near the middle
  sf::st_transform(sf::st_crs(streams_used))

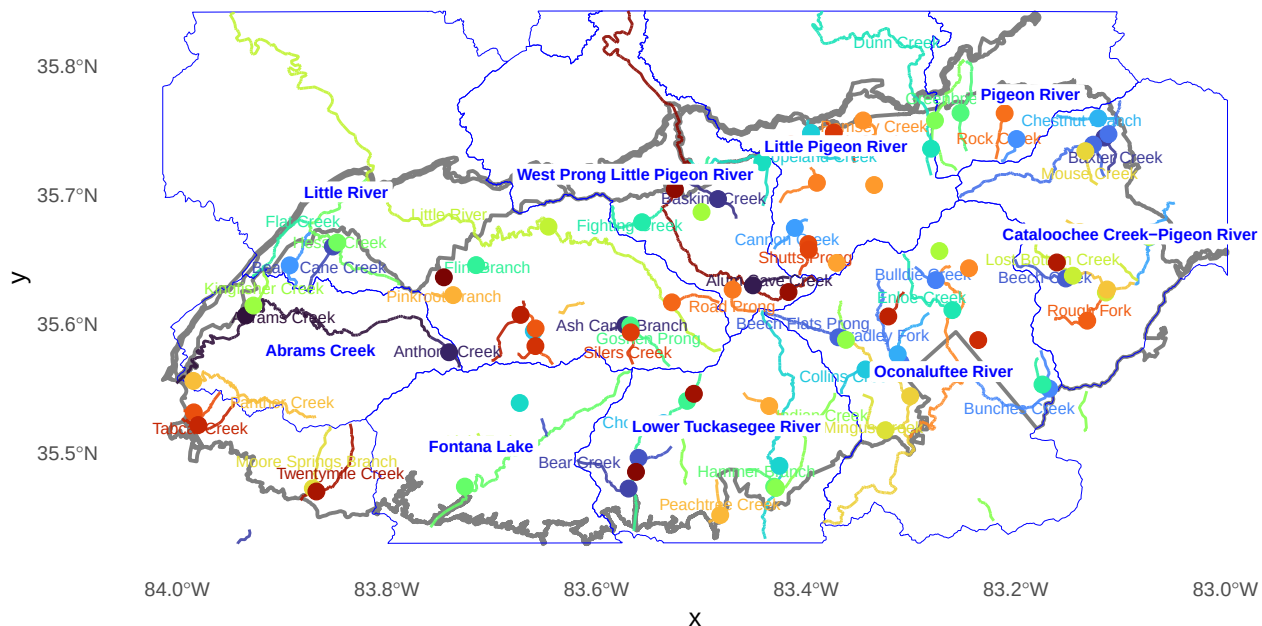
watershed_labels <- st_point_on_surface(grsm_watershed)

# 4) Plot: watersheds + border + filtered streams + locations
ggplot() +
  geom_sf(data = grsm_border, fill = NA, color = "gray50", linewidth = 1) +
  geom_sf(data = grsm_watershed, fill = NA, color = "blue", linewidth = 0.15) +
  geom_sf(data = streams_used, aes(color = Stream), linewidth = .75, alpha = 0.9) +
  geom_sf_text(data = stream_labels, aes(label = Stream, color = Stream),
    size = 3, check_overlap = TRUE) +
  geom_sf(data = sites_pts, aes(color = stream), size = 4) +
  geom_sf_label(data = grsm_watershed, aes(label = name), fill = "white", fontface = "bold", size = 3,
    label.r = unit(0.15, "lines"), size = 3) +
  scale_color_viridis_d(option = "turbo", guide = "none") +

```

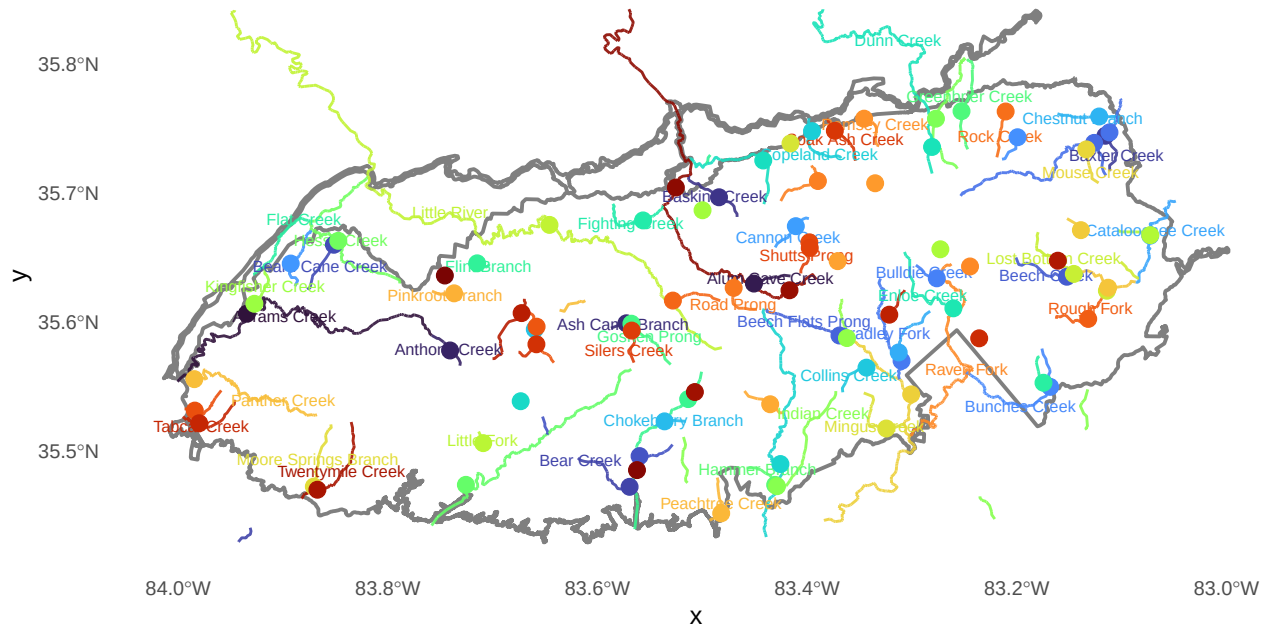
```
coord_sf() +
labs(title = "GRSM Invert Locations") +
theme_minimal(base_size = 14) +
theme(panel.grid = element_blank())
```

GRSM Invert Locations



```
# No watershed
ggplot() +
  geom_sf(data = grsm_border, fill = NA, color = "gray50", linewidth = 1) +
  #geom_sf(data = grsm_watershed, fill = NA, color = "blue", linewidth = 0.15) +
  geom_sf(data = streams_used, aes(color = Stream), linewidth = .75, alpha = 0.9) +
  geom_sf_text(data = stream_labels, aes(label = Stream, color = Stream),
    size = 3, check_overlap = TRUE) +
  geom_sf(data = sites_pts, aes(color = stream), size = 4) +
  #geom_sf_label(data = grsm_watershed, aes(label = name), fill = "white", fontface = "bold", size = 3,
  #  label.r = unit(0.15, "lines"), size = 3) +
  scale_color_viridis_d(option = "turbo", guide = "none") +
  coord_sf() +
  labs(title = "GRSM Invert Locations") +
  theme_minimal(base_size = 14) +
  theme(panel.grid = element_blank())
```

GRSM Invert Locations



```
#----- Get Slopes -----

# --- Prepare invert responses ---
inverts_stream <- inverts_stream %>%
  mutate(
    log_richness      = log(rich_stream),
    log_abundance     = log(abund_stream),
    log_EPT_richness  = log(EPT_rich_stream),
    log_EPT_abundance = log(EPT_abund_stream)
  ) %>%
  mutate(
    across(
      c(log_richness, log_abundance, log_EPT_richness, log_EPT_abundance),
      ~ ifelse(is.finite(.x), .x, NA_real_)
    )
  )

# --- Responses to analyze ---
responses <- c(
  "log_richness",
  "log_abundance",
  "log_EPT_richness",
  "log_EPT_abundance",
  "ncbi_stream" # left unlogged
)

# Optional pretty labels for plot titles
resp_labels <- c(
  log_richness      = "Invertebrate Richness Trends",
  log_abundance     = "Invertebrate Abundance Trends",
  log_EPT_richness  = "EPT Richness Trends",

```

```

log_EPT_abundance = "EPT Abundance Trends",
ncbi_stream       = "NCBI Index Trends"
)

# --- Trend helper ---
fit_stream_trend <- function(stream_data, response_var){
  m <- lm(stats::reformulate("Year", response_var), data = stream_data)
  cs <- coef(summary(m))
  i <- match("Year", rownames(cs))
  tibble(
    slope   = ifelse(is.na(i), NA_real_, cs[i, "Estimate"]),
    p_value = ifelse(is.na(i), NA_real_, cs[i, "Pr(>|t|)"]),
    r_sq    = summary(m)$r.squared
  )
}

# --- Compute per-stream trends for inverts only ---
lm_per_stream <- purrr::map_dfr(responses, function(var){
  inverts_stream %>%
    filter(n_years_sampled >= 5, is.finite(.data[[var]])) %>%
    group_by(Stream) %>%
    group_modify(~ fit_stream_trend(.x, var)) %>%
    mutate(
      response      = var,
      significant = ifelse(!is.na(p_value) & p_value < 0.05, "yes", "no")
    ) %>%
    ungroup()
})

# --- Join geometry (one geometry per Stream) ---
loc_geom <- inverts_stream_sf %>%
  group_by(Stream) %>%
  slice_head(n = 1) %>%
  dplyr::select(Stream, geometry)

lm_per_stream_sf <- lm_per_stream %>%
  left_join(loc_geom, by = "Stream") %>%
  sf::st_as_sf()

# --- Plot loop over all responses ---
gray_out <- FALSE # TRUE = gray nonsignificant, FALSE = color all

for (var in responses) {
  pts <- lm_per_stream_sf %>% filter(response == var)
  if (nrow(pts) == 0) next

  max_abs <- max(abs(pts$slope), na.rm = TRUE)
  if (!is.finite(max_abs) || max_abs == 0) max_abs <- 1

  pts <- pts %>%
    mutate(fill_val = if (gray_out) ifelse(significant == "yes", slope, NA_real_) else slope)
}

```

```

p <- ggplot() +
  geom_sf(data = grsm_border, fill = NA, color = "gray50", linewidth = 1) +
  geom_sf(data = streams_used, color = "grey70", linewidth = 0.5, alpha = 0.7) +
  geom_sf_text(data = stream_labels, aes(label = Stream), color = "black",
    size = 3, check_overlap = TRUE, show.legend = FALSE) +
  geom_sf(data = pts, aes(fill = fill_val),
    shape = 21, color = "black", size = 7, stroke = 0.3) +
  scale_fill_gradientn(
    colors = c("#1E90FF", "white", "red"),
    limits = c(-max_abs, max_abs),
    na.value = "grey70",
    name = "Slope"
  ) +
  coord_sf() +
  labs(title = resp_labels[[var]]) +
  theme_minimal(base_size = 14) +
  theme(panel.grid = element_blank())

print(p)
}

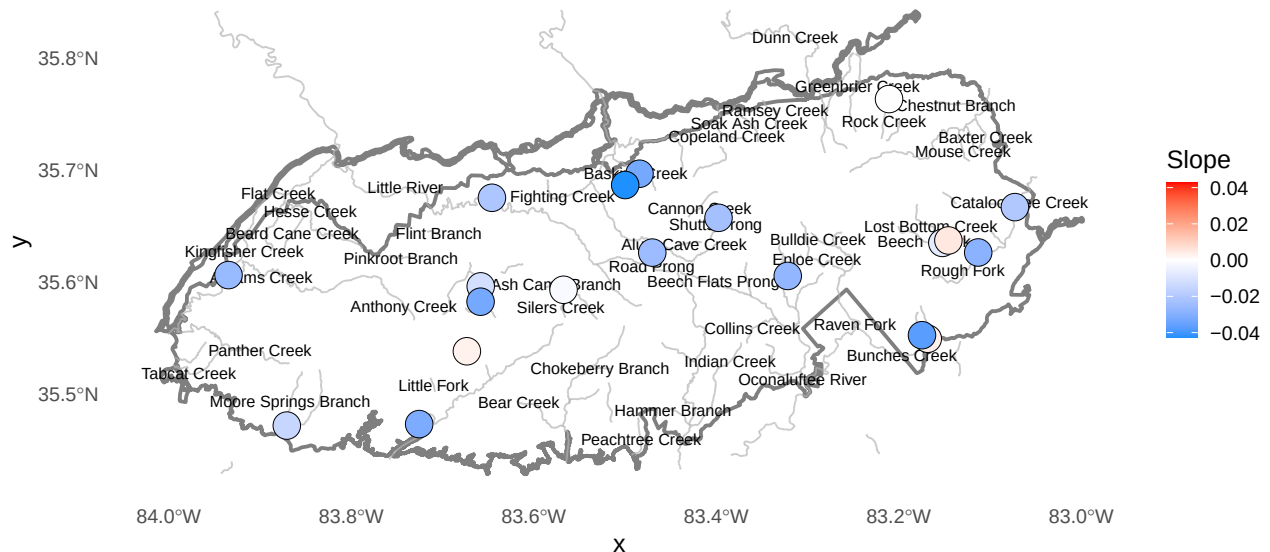
```

```

## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not
## give correct results for longitude/latitude data

```

Invertebrate Richness Trends

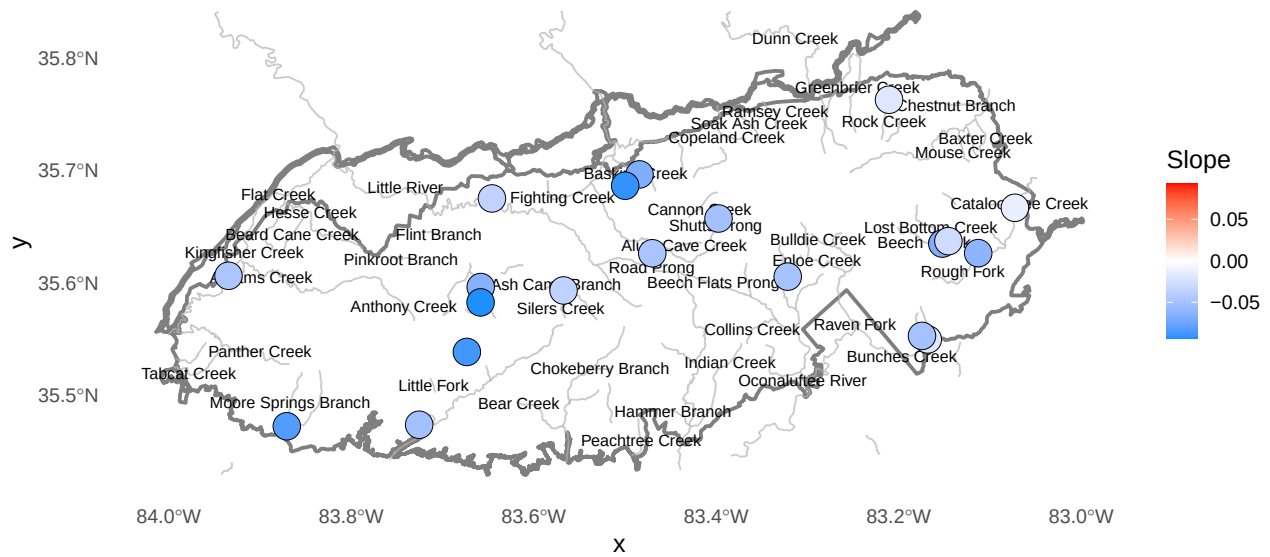


```

## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not
## give correct results for longitude/latitude data

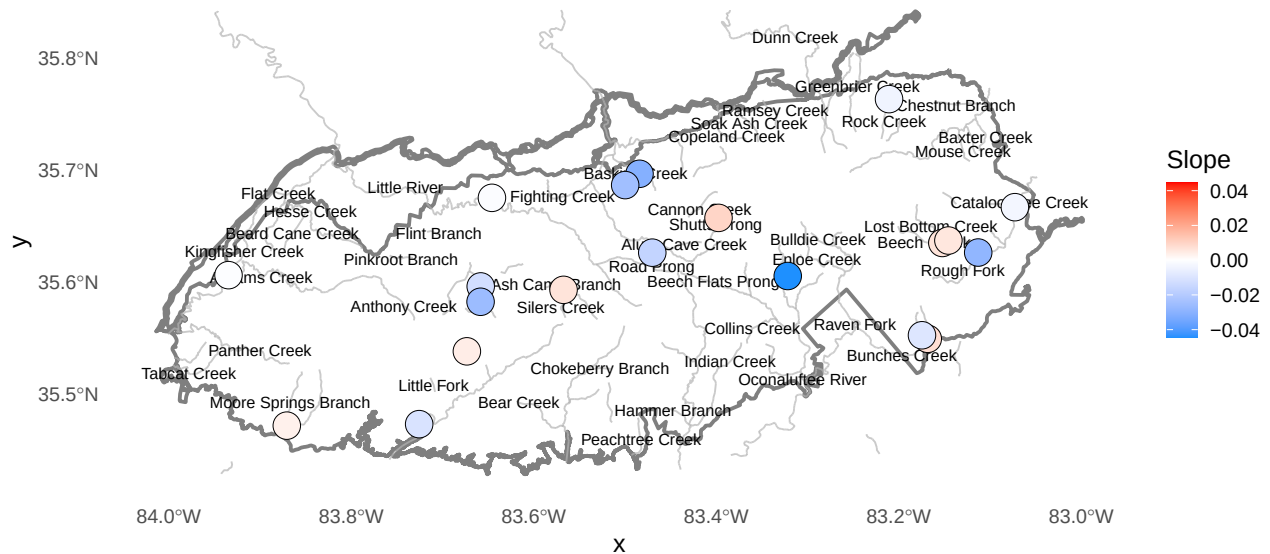
```

Invertebrate Abundance Trends



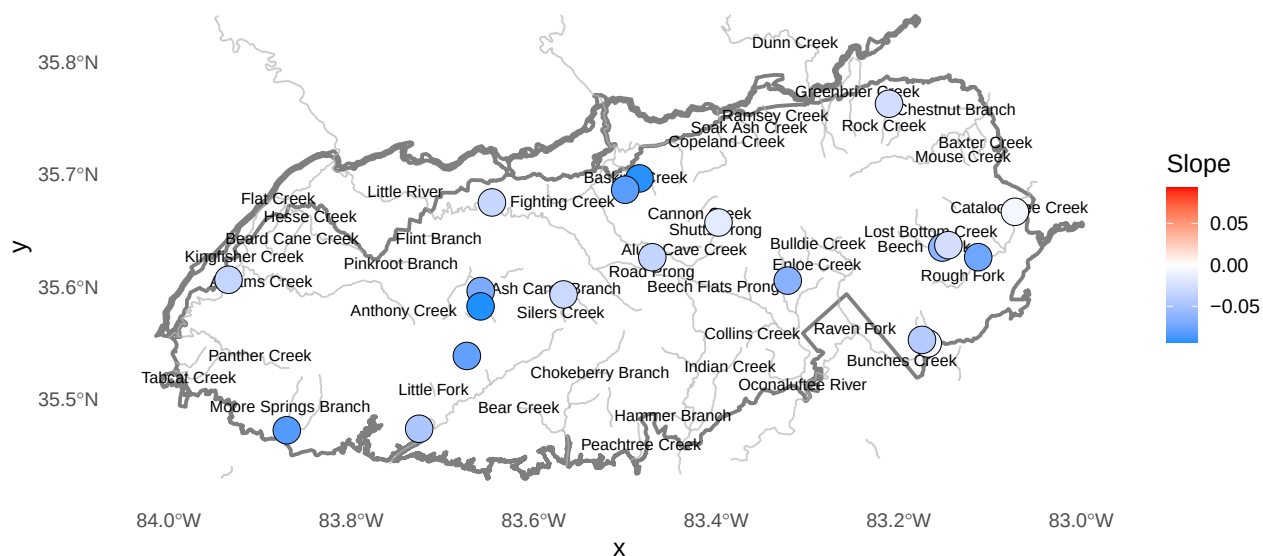
```
## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not
## give correct results for longitude/latitude data
```

EPT Richness Trends



```
## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not
## give correct results for longitude/latitude data
```

EPT Abundance Trends



```
## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not
## give correct results for longitude/latitude data
```

NCBI Index Trends

